

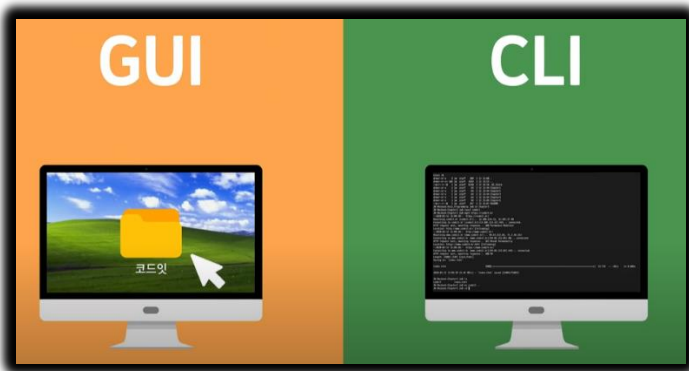
Contents

WINDOWS COMMANDLINE PART 1.....	2
STANDARD WINDOWS UTILITIES	7
COMMANDLINE SWITCHES AND PIPES	20
COMMAND LINE SYNTAX	23
PARAMETERS.....	26
WHY USE BATCH FILES?	29
COMMAND-LINE UTILITIES BY PURPOSE	39
MICROSOFT UTILITIES UPDATES.....	48
USER ACCOUNT CONTROL, RUNAS	55

WINDOWS COMMANDLINE PART 1

The Command Line's Power:

- Remains a powerful tool for administrators and users
- GUI is convenient but limited
- Some tasks aren't accessible through the GUI
- Relying on the GUI can be inefficient and error-prone



Automation and Efficiency:

- Enables task automation
- Saves time on repetitive work
- Reduces the chance of human error
- Allows quick execution of multiple tasks without manual GUI steps



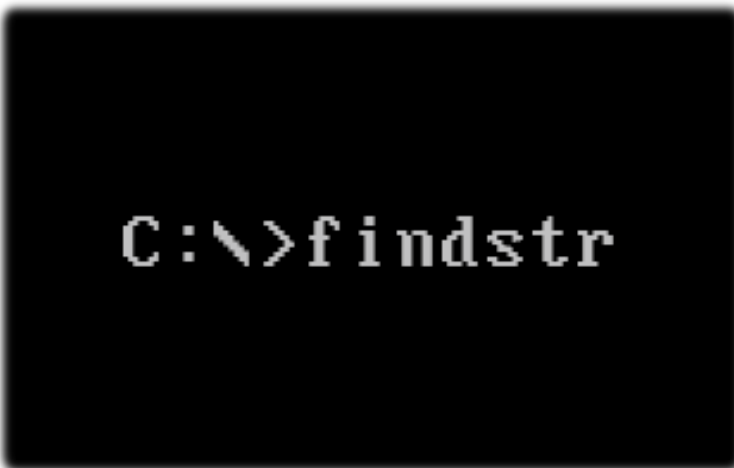
Windows PowerShell vs Command Line:

- PowerShell is the primary and actively developed shell in modern Windows
- Command Prompt is still included and supported for compatibility
- Recent Windows versions continue to enhance PowerShell (and introduce cross-platform PowerShell Core)
- The book's discussion can be mapped to how these tools coexist and evolve in current Windows versions



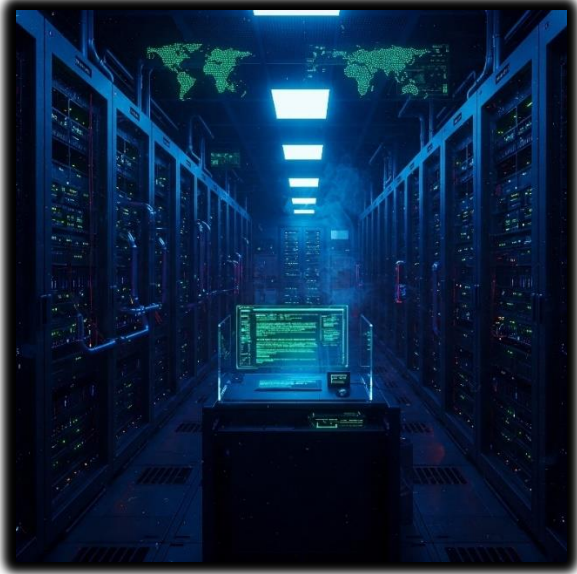
Why the Command Line is Better:

- GUI search is often limited or inefficient for specific files and tasks
- Searching for executables is especially weak in the GUI
- Command-line tools provide faster, more precise searches
- Utilities like **FindStr** enable high-speed searching the GUI can't match



Detailed Information and Real-World Use:

- This guide goes beyond basic command usage with in-depth explanations, tips, and real-world scenarios
- Makes commands more practical and useful, not just theoretical
- Suitable for both beginners and experienced users
- Includes a beginner-friendly Chapter 1 for those new to the command line



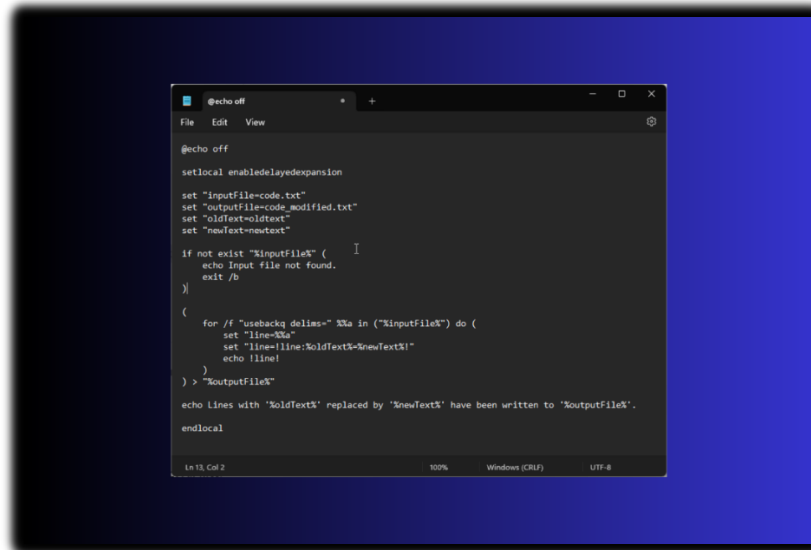
Real World Scenarios:

- Commands and utilities are supported by real-world examples
- Explains how and why to use each tool
- Highlights common pitfalls and mistakes to avoid



Scripts and Batch Files:

The author provides scripts and batch files based on personal experience, which readers can use or adapt for their needs. These tools are presented as real-world, tested solutions.



```
@echo off
setlocal enabledelayedexpansion

set "inputFile=code.txt"
set "outputFile=code_modified.txt"
set "oldText=oldtext"
set "newText=newtext"

if not exist "%inputFile%" (
    echo Input file not found.
    exit /b
)

for /f "usebackq delims=" %a in ("%inputFile%") do (
    set "line=%a"
    set "line=line:%oldText%>newText%"
    echo !line!
) > "%outputFile%"

echo Lines with "%oldText%" replaced by "%newText%" have been written to "%outputFile%".

endlocal
```

THE BOOK AND WHAT YOU'LL LEARN

Broad Appeal (Chapters 1-5):

- The initial chapters are designed for anyone who uses Windows, regardless of their technical expertise.
- These chapters will introduce powerful command-line tools that can simplify everyday tasks.



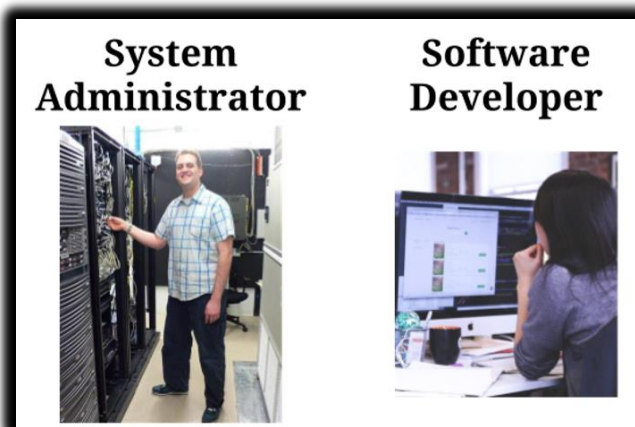
Basic Windows Knowledge Assumed:

- Early chapters are intended for all readers
- Assumes basic Windows knowledge (file navigation, keyboard/mouse use)
- Does not cover Windows fundamentals
- Jumps straight into command-line usage



Developer and Administrator Focus (Chapters 6-17):

- Covers advanced command-line topics
- Includes tasks not possible through the GUI
- Explains accessing functions in DLL files
- Focuses on task automation
- Introduces useful third-party tools
- Explores .NET Framework features (e.g., compiling JScript)



Looking Ahead to Vista and PowerShell (Chapters 18–19):

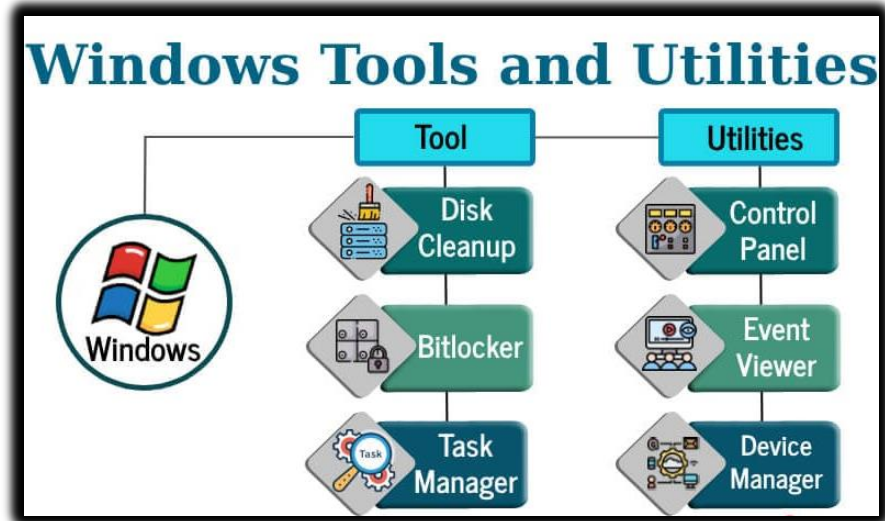
- Explores the command-line experience in Windows Vista
- Introduces Windows PowerShell as a major advancement
- Highlights PowerShell's power, efficiency, and improved security
- Notes similarities to UNIX shells
- Mentions that PowerShell can be installed on older Windows versions



STANDARD WINDOWS UTILITIES

Standard Windows Utilities — Chapter Breakdown

- **Ch. 1:** Using the Command Line Effectively
- **Ch. 2:** Completing Data-Specific Tasks
- **Ch. 3:** Discovering the System Status
- **Ch. 4:** Locating Files & Other Resources
- **Ch. 5:** Securing & Monitoring a System
- **Ch. 6:** Developer & Low-Level Utilities



Chapter 1: Using the Command Line Effectively

Why It Matters

- Command line = powerful tool; GUI = convenient but limited
- CLI enables faster work, automation, and fewer mistakes
- Many critical system features are accessible only via CLI (including PowerShell)
- Modern Windows CLI is more scriptable, robust, and user-friendly than older versions
- "It's not outdated—it's underground"

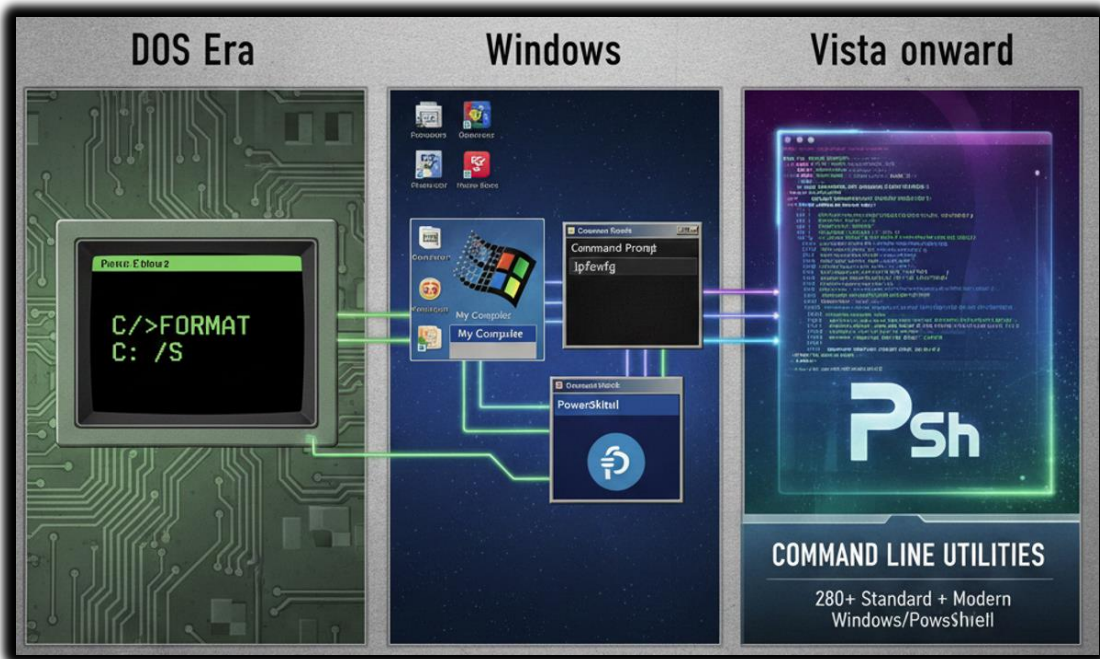
Why People Dropped It

- GUI made computers more approachable, but didn't replace all tasks
- CLI may seem harder (syntax, memorization), but scripting and tab completion make it easier today
- Many tasks remain faster, more precise, or only possible via CLI



Evolution of the Command Line

- **DOS era:** Full command-line environment
- **Windows:** GUI became dominant, but CLI (Command Prompt & PowerShell) remained supported
- **Vista onward:** Introduced new CLI enhancements and tools; PowerShell emerged as the primary advanced shell
- Book covers ~280 utilities (standard + modern Windows/Powershell-specific)



💡 What You Can Do with It

- Automate tasks using batch files or PowerShell scripts (no need to memorize commands every time)
- Run commands in the background, on schedule, or without user interaction
- Replace third-party tools with native utilities (e.g., **FindStr**, **Dir**, **Get-ChildItem** in PowerShell)

🔧 Tools & Concepts Introduced in Chapter 1

- Organizing commands by purpose rather than alphabetically
- **Internal vs External commands:** Internal = built into cmd.exe (e.g., `dir`, `cd`); External = separate executables
- Customizing the command window (font, size, colors, scrollbar, etc.)
- Updating utilities via official Microsoft sources or PowerShell modules

- Modern differences in commands/parameters across Windows versions, including PowerShell equivalents



📌 Takeaways

- CLI provides efficiency, power, and control
- Scripts and batch files save time and reduce repetition
- Modern Windows includes a rich set of built-in tools; the book helps you discover and leverage them

📌 Understanding Why the Command Line Is So Important

📖 A Little History

🌟 Windows was born to:

- ✓ Allow multitasking (DOS was one-app-at-a-time).
- ✓ Be more “*user-friendly*” (no need to memorize commands).

- GUI succeeded in making interactive apps easier (word processors, dev tools).
- But many **noninteractive/background tasks** are still better handled with CLI.



✂ When GUI Just Ain't It

- Example: **Backups**.
 - ✓ Should run in background.
 - ✓ Need speed, accuracy, and repeatability.
 - ✓ GUI = not ideal (you're using files, it slows down, and not easily automated).
 - ✓ CLI = perfect (no need for user input, can run scheduled).
- CLI > GUI for **automation**.

🔍 Real Talk: Search is Broken in Explorer

- GUI search is **slow, inaccurate**, and **doesn't index** everything (even on Vista).
- Want to search inside binaries or random file types? GUI fails.
- CLI tool like FindStr** crushes it.
 - ✓ Searches **inside any file**, including executables.
 - ✓ Fast, direct, and accurate.



Security: What's Actually Running?

GUI's Task Manager:

- Limited view (SvcHost entries are vague).
- Even Window's "Services" tab helps *a little*.

Apps (11)						
> Balabolka (32 bit)	0%	23.9 MB	0 MB/s	0 Mbps	0%	
> Clover (32 bit)	0.2%	2.9 MB	0 MB/s	0 Mbps	0%	
> FocusX.v3.exe	0%	2.7 MB	0 MB/s	0 Mbps	0%	
> GitHubDesktop.exe (2)	0.6%	101.4 MB	0 MB/s	0.1 Mbps	0%	
> Google Chrome (32)	0.3%	2,115.6 MB	0.1 MB/s	0 Mbps	0.1%	GPU 0 - 3D
> Microsoft Word	0%	146.6 MB	0 MB/s	0 Mbps	0%	
> Photos (5)	0.7%	316.8 MB	0 MB/s	0 Mbps	0%	GPU 0 - 3D
> Sublime Text (4)	0%	43.6 MB	0 MB/s	0 Mbps	0%	
> SumatraPDF	0%	100.7 MB	0 MB/s	0 Mbps	0%	
> Task Manager	2.0%	26.5 MB	0 MB/s	0 Mbps	0%	
> Vivaldi (15)	0.2%	917.8 MB	0 MB/s	0 Mbps	0%	GPU 0 - 3D
Background processes (91)						

CLI's TaskList:

- Shows exact services running **inside** each SvcHost.
- Reveals **PID** (Process ID) so you can trace what's really happening.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Nick The Chief> tasklist /svc

Image Name                      PID Services
=====
System Idle Process             0 N/A
System                          4 N/A
Registry                        108 N/A
smss.exe                        480 N/A
csrss.exe                       632 N/A
wininit.exe                     804 N/A
csrss.exe                       812 N/A
services.exe                   876 N/A
lsass.exe                       888 EFS, KeyIso, SamSs, VaultSvc
winlogon.exe                    984 N/A
svchost.exe                     688 BrokerInfrastructure, DcomLaunch, PlugPlay,
Power, SystemEventsBroker
fontdrvhost.exe                 816 N/A
fontdrvhost.exe                 504 N/A
WUDFHost.exe                    1084 N/A
svchost.exe                     1164 RpcEptMapper, RpcSs
svchost.exe                     1212 LSM
svchost.exe                     1304 lmhosts
svchost.exe                     1364 NcbService
svchost.exe                     1376 TimeBrokerSvc
svchost.exe                     1408 EventLog
svchost.exe                     1488 hidserv
dwm.exe                         1528 N/A
svchost.exe                     1616 nsi
svchost.exe                     1672 Dhcp
svchost.exe                     1696 SEMgrSvc
svchost.exe                     1816 Schedule
svchost.exe                     1880 ProfSvc
svchost.exe                     1940 NlaSvc
svchost.exe                     2044 UserManager
atiesrxx.exe                    1148 AMD External Events Utility
svchost.exe                     1748 camsvc
svchost.exe                     2084 CoreMessagingRegistrar
svchost.exe                     2092 netprofm
helperservice.exe               2200 N/A
svchost.exe                     2224 StateRepository
svchost.exe                     2344 DispBrokerDesktopSvc
conhost.exe                     2432 N/A
svchost.exe                     2440 WinHttpAutoProxySvc
svchost.exe                     2536 Dnscache
atieclxx.exe                    2632 N/A

```

🧠 You can't defend your system if you don't know what it's doing. CLI tells the truth, GUI hides it.

CLI: Your X-Ray Goggles for Windows

Network & Web Intelligence

- **DNS resolution:** Use CLI to find exact IP addresses behind URLs
 -  Example: `nslookup openai.com` helps identify spoofed or suspicious sources
- **Diagnose slow internet:** Determine if issues are with Wi-Fi, ISP, or network routes
 -  Tools: `ping`, `tracert`, `pathping` provide quick, accurate diagnostics before contacting support

```
PS C:\Users\Nick The Chief> nslookup telegram.org
Server:  UnKnown
Address:  192.168.110.1

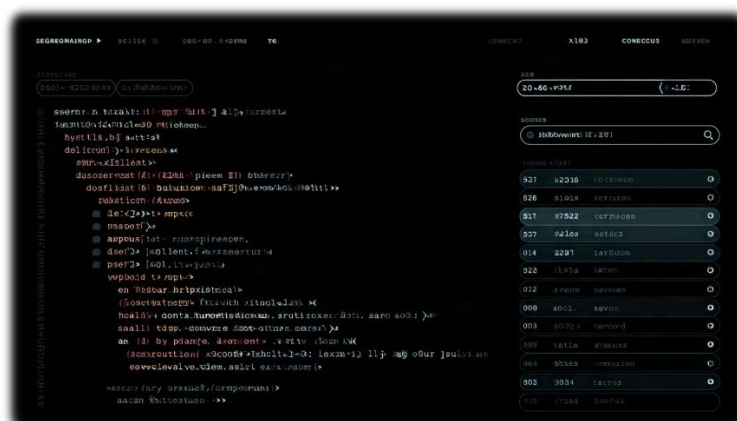
Non-authoritative answer:
Name:     telegram.org
Addresses: 2001:67c:4e8:f004::9
          149.154.167.99

PS C:\Users\Nick The Chief> nslookup nick.com
Server:  UnKnown
Address:  192.168.110.1

Non-authoritative answer:
Name:     nick.com
Addresses: 2600:1f18:297:ba24:b508:76de:f4a:df34
          2600:1f18:297:ba0e:7900:f622:48a8:f266
```

Local + Network Resource Discovery

- CLI doesn't just talk to the web—it **maps your local network** too.
 -  `netstat`, `ipconfig`, `nbtstat`, `arp`, and friends help you find rogue devices, hidden services, and weird traffic.
 -  Forget the GUI's "Network" tab—it only shows happy paths. CLI shows **everything**, including what *shouldn't* be there.



Deep System Visibility

- The command line sees stuff the GUI pretends doesn't exist. You want actual disk usage? Real-time CPU info? Driver status? It's all there.
- CLI **doesn't assume** what you're trying to do—it just **does what you tell it**.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Nick The Chief> fsutil volume diskfree C:
Total free bytes       : 109,845,753,856 (102.3 GB)
Total bytes            : 179,373,850,624 (167.1 GB)
Total quota free bytes : 109,845,753,856 (102.3 GB)
PS C:\Users\Nick The Chief> wmic cpu get LoadPercentage
LoadPercentage
6

PS C:\Users\Nick The Chief> driverquery /v
```

Module Name	Display Name	Description	Driver Type	Start Mode	State	Status	Accept Stop	Accept Pause	Paged Pool(bytes)
Path			Init(bytes)						
1394ohci	1394 OHCI Compliant Ho	1394 OHCI Compliant Ho	Kernel	Manual	Stopped	OK	FALSE	FALSE	4,096
	C:\Windows\system32\drivers\1394ohci.sys		4,096						
3ware	3ware	3ware	Kernel	Manual	Stopped	OK	FALSE	FALSE	0
	C:\Windows\system32\drivers\3ware.sys		4,096						
ACPI	Microsoft ACPI Driver	Microsoft ACPI Driver	Kernel	Boot	Running	OK	TRUE	FALSE	176,128
	C:\Windows\system32\drivers\ACPI.sys		24,576						
AcpiDev	ACPI Devices driver	ACPI Devices driver	Kernel	Manual	Stopped	OK	FALSE	FALSE	8,192
	C:\Windows\system32\drivers\AcpiDev.sys		4,096						
acpiex	Microsoft ACPIEx Drive	Microsoft ACPIEx Drive	Kernel	Boot	Running	OK	TRUE	FALSE	40,960
	C:\Windows\system32\Drivers\acpiex.sys		4,096						
acpipagr	ACPI Processor Aggrega	ACPI Processor Aggrega	Kernel	Manual	Stopped	OK	FALSE	FALSE	4,096
	C:\Windows\system32\drivers\acpipagr.sys		4,096						

Why CLI Is Essential (Even If You Don't Use It Daily)

GUI

Babysits you with buttons

Shows what it thinks you *need*

Slows down with animations

Hides complex routes & internals

CLI

Treats you like an admin

Shows **everything**, even the risky stuff

Lightning-fast execution, no graphics to slow it down

Exposes true paths, ports, and performance

⚠ Reality Check

- For serious system security or performance, the GUI alone isn't enough
- The CLI remains fully supported and powerful
- You don't need to use it constantly, but you should know it thoroughly—like a reliable sidearm
- Modern CLI (Command Prompt + PowerShell) gives access to tools and capabilities the GUI can't match

Command Line is NOT That Deep

🤖 What People Think the CLI Is:

"You just type a bunch of cryptic stuff, hope you don't brick your machine, then stare at white text on a black screen like a hacker in a bad movie."

Wrong. That's the **myth**.



📖 CLI Is Easy If You Know the Secret Knock

You don't need to memorize commands like it's the SAT. You need **just one trick**:

Add `/?` to any command to get instant help. That's it. 🗝

- `tasklist /?`
- `xcopy /?`
- `netstat /?`
- `shutdown /?`

Almost *every* built-in Windows CLI tool responds to that magic switch and dumps a cheat sheet right in your terminal.

You'll see all the **options** the command supports (scrollable if it's long).

🌟 Why You Should Use Commands Directly First

Yeah, scripting and batch files are cool...

But before you automate anything, you've gotta understand:

- What each command does
- What input it needs
- What output it gives

Once you *get* a few commands, stringing them together into automation is child's play.

- Knowing how to use command `/?` to read help puts you ahead of most Windows users
- Next step: organize commands by purpose (file operations, networking, diagnostics, etc.)
- This approach helps you quickly find the right tool without guessing
- Builds a clear mental map of Windows CLI capabilities

```
PS C:\Users\Nick The Chief> tasklist /?

TASKLIST [/S system [/U username [/P [password]]]]
          [/M [module] | /SVC | /V] [/FI filter] [/FO format] [/NH]

Description:
  This tool displays a list of currently running processes on
  either a local or remote machine.

Parameter List:
  /S      system          Specifies the remote system to connect to.
  /U      [domain\]user   Specifies the user context under which
                          the command should execute.
  /P      [password]      Specifies the password for the given
                          user context. Prompts for input if omitted.
  /M      [module]        Lists all tasks currently using the given
                          exe/dll name. If the module name is not
                          specified all loaded modules are displayed.
  /SVC                      Displays services hosted in each process.
  /APPS                      Displays Store Apps and their associated processes.
  /V                      Displays verbose task information.
  /FI      filter          Displays a set of tasks that match a
                          given criteria specified by the filter.
  /FO      format          Specifies the output format.
                          Valid values: "TABLE", "LIST", "CSV".
  /NH                      Specifies that the "Column Header" should
                          not be displayed in the output.
                          Valid only for "TABLE" and "CSV" formats.
  /?                      Displays this help message.
```

```
PS C:\Users\Nick The Chief> netstat /?
```

Displays protocol statistics and current TCP/IP network connections.

```
NETSTAT [-a] [-b] [-e] [-f] [-n] [-o] [-p proto] [-r] [-s] [-t] [-x] [-y] [interval]
```

-a	Displays all connections and listening ports.
-b	Displays the executable involved in creating each connection or listening port. In some cases well-known executables host multiple independent components, and in these cases the sequence of components involved in creating the connection or listening port is displayed. In this case the executable name is in [] at the bottom, on top is the component it called, and so forth until TCP/IP was reached. Note that this option can be time-consuming and will fail unless you have sufficient permissions.
-e	Displays Ethernet statistics. This may be combined with the -s option.
-f	Displays Fully Qualified Domain Names (FQDN) for foreign addresses.
-n	Displays addresses and port numbers in numerical form.
-o	Displays the owning process ID associated with each connection.
-p proto	Shows connections for the protocol specified by proto; proto may be any of: TCP, UDP, TCPv6, or UDPv6. If used with the -s option to display per-protocol statistics, proto may be any of: IP, IPv6, ICMP, ICMPv6, TCP, TCPv6, UDP, or UDPv6.
-q	Displays all connections, listening ports, and bound nonlistening TCP ports. Bound nonlistening ports may or may not be associated with an active connection.
-r	Displays the routing table.
-s	Displays per-protocol statistics. By default, statistics are shown for IP, IPv6, ICMP, ICMPv6, TCP, TCPv6, UDP, and UDPv6; the -p option may be used to specify a subset of the default.
-t	Displays the current connection offload state.
-x	Displays NetworkDirect connections, listeners, and shared endpoints.
-y	Displays the TCP connection template for all connections. Cannot be combined with the other options.
interval	Redisplays selected statistics, pausing interval seconds between each display. Press CTRL+C to stop redisplaying statistics. If omitted, netstat will print the current configuration information once.

COMMANDLINE SWITCHES AND PIPES

Command-Line Switches

- A **switch** is an option or modifier added to a command to change its behavior or output
- Think of it as giving extra instructions to the main command
- Switches let you customize the command's actions or output
- **Syntax:** Usually starts with a **forward slash (/)** or sometimes a **dash (-)**, followed by a letter or word

Example: dir Command

- Typing dir alone shows a basic file/folder listing
- Add switches to get more specific output:
 - ✚ /W → "Wide" format listing
 - ✚ /A → Filter by file attributes
 - Example: /A:H shows **hidden files**
 - ✚ /O → Sort the output (e.g., /O:N sorts by name)
 - ✚ /S → Include subdirectories
 - ✚ /P → Pause output one screen at a time
- Combining switches gives even more control:
 - ✚ Example: dir /A:H /O:N /P → Shows hidden files, sorted by name, pausing each screen
- Modern tip: PowerShell uses **parameters** instead of switches, often with a - prefix (e.g., Get-ChildItem -Hidden -Recurse)

```

C:\Users\Nick The Chief\Downloads>dir /a:h
Volume in drive C has no label.
Volume Serial Number is BCC5-4691

Directory of C:\Users\Nick The Chief\Downloads

04/09/2025  04:19 PM                282 desktop.ini
04/11/2025  07:31 PM                162 ~$eManOfSteel2 - NixonSchool92.docx
04/11/2025  03:12 PM                162 ~$eWorld.docx
               3 File(s)                606 bytes
               0 Dir(s)  109,825,507,328 bytes free

C:\Users\Nick The Chief\Downloads>dir /a:s
Volume in drive C has no label.
Volume Serial Number is BCC5-4691

Directory of C:\Users\Nick The Chief\Downloads

04/09/2025  04:19 PM                282 desktop.ini
               1 File(s)                282 bytes
               0 Dir(s)  109,825,445,888 bytes free

C:\Users\Nick The Chief\Downloads>dir /a
Volume in drive C has no label.
Volume Serial Number is BCC5-4691

Directory of C:\Users\Nick The Chief\Downloads

04/12/2025  07:49 AM    <DIR>          .
04/12/2025  07:49 AM    <DIR>          ..
11/11/2024  07:44 AM    31,133,896 1 ... Operating Systems.pdf
11/02/2024  11:51 AM    13,946,903 2 ... TCP-IP-Vol1.pdf
11/02/2024  11:52 AM    39,820,078 3 ... TCP-IP-Vol2.pdf
11/30/2024  04:59 PM    16,110,511 4 ... CommandLine Administration Best2.pdf

```

Pipes (|):

- A **pipe (|)** sends the output of one command as input to another command
- Think of it as connecting two commands: the first command produces data, the second command processes it
- Useful for filtering, sorting, or processing data without creating temporary files

Example: List all files in a directory **containing "Gemini"** in their name.

```

C:\Users\Nick The Chief\Downloads>dir | findstr "Gemini"
04/11/2025  11:44 PM    539,008 Gemini_Generated_Image_idzoulidzoulidzo.jpg
04/11/2025  11:45 PM    630,299 Gemini_Generated_Image_lffprlffprlffprl.jpg

```

Pipes can chain multiple commands for complex operations:

Example: `dir | find "Gemini" | sort` → Lists, filters, and sorts results in one line

Command-Line Switches

- Switches modify the behavior of a single command
- Typically start with / (Command Prompt) or - (PowerShell/Unix-style)
- Switches / options use the forward slash / (not the dash - like in PowerShell, Linux, macOS, etc.)

```
dir /s /b /a-d
```

- /s → include subdirectories
- /b → bare format (just names, no extra info)
- /a-d → show files only (exclude directories)

Pipe with:

```
dir | find "2025"
```

- dir lists files/folders
- sends that list to find
- find "2025" shows only lines containing "2025" (e.g., files from 2025)

Combining both:

```
ipconfig /all | findstr /i "ipv4 physical dns"
```

- ipconfig /all → shows full network configuration
- pipes it to findstr
- findstr /i "ipv4 physical dns" → case-insensitive search for those keywords (shows IP addresses, MAC, DNS servers, etc.)

Quick comparison table

Feature	Windows CMD	PowerShell / *nix
Switch Prefix	/	- or --
Pipe Symbol		
Example Switch	dir /p	ls -l
Pipe Example	dir more	ls grep .txt

COMMAND LINE SYNTAX ⚡

Most Windows CLI commands follow this format:

```
Command [options] [arguments]
```

Understanding Optional Parameters (Square Brackets [])

In command syntax, **square brackets** [] mean that the enclosed parameter is **optional**—you can include it or omit it.

Example:

```
C:\Users\Nick The Chief\Downloads>tasklist [/S system [/U username [/P [password]]]]
```

Commands can be run with or without optional parameters

Square brackets [] in our image above here, indicate **optional parts**

Do **not** type the brackets themselves

Example with tasklist:

- /S system → optional
- /U username → optional, only if /S is used
- /P password → optional, only if /U is used

Brackets are **documentation shorthand**, not literal characters to type

✅ Correct Ways to Run It:

1. Basic Usage (no options)

cmd

Copy

```
tasklist
```

- Lists all running processes on your current machine.

2. With a Remote System (/S)

cmd

Copy

```
tasklist /S 192.168.1.100
```

- Lists processes on a remote computer (IP/hostname).

3. With Username (/U)

cmd

Copy

```
tasklist /S 192.168.1.100 /U Admin
```

- Connects to `192.168.1.100` using the username `Admin`.

4. With Password (/P)

cmd

Copy

```
tasklist /S 192.168.1.100 /U Admin /P MyPassword123
```

- Provides the password for authentication.

5. Password Prompt (if `/P` is empty)

cmd

Copy

```
tasklist /S 192.168.1.100 /U Admin /P
```

- You'll be **prompted** to enter the password (more secure).

You can do this with Virtual Machines which is a simpler way to explore. Let's do it, I have some VMs ready.

✗ Incorrect Ways (What NOT to Do)

- Including `[ ]` in the command:

cmd

Copy

```
tasklist [/S 192.168.1.100] ✗ (Wrong!)
```

- The brackets are just notation—don't type them!

- Skipping required parts of a switch:

cmd

Copy

```
tasklist /U Admin ✗ (Missing /S system)
```

- If you use `/U`, you **must** have `/S` first.

`[ ]` indicates an **optional switch** (include or exclude as needed)

Do not type `[ ]` in the actual command

Order matters:

- `/S` must come before `/U`
- `/U` must come before `/P`

If `/P` is used without a password, you will be **prompted**

Windows Command Prompt prefers `/` over `-` like Linux

PowerShell often **accepts both `/` and `-`**

PARAMETERS

- **Parameters** are values given to a command **without using / or -** like switches
- They tell the command **what to act upon**
- Some commands accept **drive letters** (e.g., D:) as parameters to specify the target storage location

```
C:\Users\Nick The Chief\Downloads>dir D:
Volume in drive D is ToshibaX
Volume Serial Number is F8D9-AC95

Directory of D:\

11/11/2024  07:44 AM                31,133,896 1 ... Operating Systems.pdf
11/02/2024  11:51 AM                13,946,903 2 ... TCP-IP-Vol1.pdf
11/02/2024  11:52 AM                39,820,078 3 ... TCP-IP-Vol2.pdf
03/02/2025  02:30 AM                17,839,384 5 ... Automate the Boring Stuff with Python.pdf
01/31/2025  03:10 PM               294,288,017 9 Uncommon Ways to GET and STAY Rich (Watch and Share This many times).mp4
04/08/2025  02:33 PM                1,166,138 A-Farewell-to-Virology-Expert-Edition.pdf
04/11/2025  06:53 PM                <DIR>      Action!
05/15/2024  01:35 AM               518,817,795 Best of Emachichi,3 Hours Nonstop Mix, Speech Entertainment.mp4
03/19/2025  05:08 PM               228,434,309 calm your heart..mp4
04/09/2025  03:47 AM                926,454 DevonIsFreed.png
04/04/2025  12:32 AM               2,947,857,762 Dune.2021.1080p.WEBRip.x264.AAC5.1-[YTS.MX].mp4
04/04/2025  12:37 AM               2,968,283,738 Dune.Part.Two.2024.1080p.WEBRip.x264.AAC-[YTS.MX].mp4
03/25/2025  04:47 AM                2,722,103,296 EternalNotes0.hc
03/31/2025  01:30 AM                3,246,391,296 EternalNotes1.hc
03/31/2025  07:56 PM                1,073,741,824 EternalNotes2.hc
03/31/2025  07:56 PM                1,073,741,824 EternalNotes3.hc
03/31/2025  11:20 PM                2,357,198,848 EternalNotes5.hc
04/11/2025  06:53 PM                <DIR>      EverybodyHatesChris
04/01/2025  06:05 PM                6,442,450,944 FamilyPhotos.hc
04/11/2025  03:49 AM                <DIR>      GeminiAI.Images
04/09/2025  03:14 PM                <DIR>      Github.New
```

Here, **D:** is the parameter telling the **cd** command *where* you want to go.

If you want to see the files and folders at the root of your D: drive, you can use:

```
C:\Users\Nick The Chief\Downloads>D:
D:\>dir | findstr ".hc"
03/25/2025  04:47 AM                2,722,103,296 EternalNotes0.hc
03/31/2025  01:30 AM                3,246,391,296 EternalNotes1.hc
03/31/2025  07:56 PM                1,073,741,824 EternalNotes2.hc
03/31/2025  07:56 PM                1,073,741,824 EternalNotes3.hc
03/31/2025  11:20 PM                2,357,198,848 EternalNotes5.hc
04/01/2025  06:05 PM                6,442,450,944 FamilyPhotos.hc
04/11/2025  12:35 AM               15,305,015,296 HT.hc
01/31/2025  06:03 PM                209,715,200 Password.hc
03/31/2025  07:48 PM               364,904,448 The.727.System.hc

D:\>
```

✓ Good Starter Commands to Try `/?" On

Command	Use Case
<code>tasklist /?</code>	See active processes and services
<code>dir /?</code>	File and folder listings
<code>netstat /?</code>	Network ports and connections
<code>shutdown /?</code>	Restart, logoff, shutdown, timed power off
<code>robocopy /?</code>	Advanced file copy (better than <code>xcopy</code>)

Back to parameters (Values we pass to the actual commands).

1. Path Parameters

```
dir C:\Users\Rennex\Documents
```

No / is used—just the raw path.

2. Filename/Extension Parameter

```
dir *.txt      (Lists all .txt files)
dir report.doc (Looks for "report.doc")
```

`*.txt` and `report.doc` are **parameters** (filtering files).

3. Drive Letter Parameter

```
dir D:\      (Lists files on the D: drive)
```

D:\ is a **parameter** (the drive to scan).

Key Differences Between Parameters and Switches

Feature	Parameter	Switch
Syntax	No prefix (e.g., <code>C:\</code> , <code>*.exe</code>)	Prefixed with <code>/</code> (e.g., <code>/W</code> , <code>/A:H</code>)
Purpose	Tells the command <i>what</i> to act on	Tells the command <i>how</i> to act
Usage Example	<code>dir C:\Photos</code> (target folder)	<code>dir /S</code> (include subdirectories)
Acts As	Input (location, file type, etc.)	Modifier (affects behavior/output)
Required?	Usually yes (you need to point somewhere)	Often optional (enhances control)

Parameters

- What they are: The *where* or *what* the command is working on.
- Purpose: They give the command the **actual value** or **target** it should operate on.
- Think of them as: The destination you're pointing to.
- Example:

```
dir D:\Downloads
```

► “Show me what’s inside the Downloads folder.”

 **Parameter = destination.**

🔥 Switches

- **What they are:** The *how* the command should behave.
- **Purpose:** They *modify* or *enhance* what the command does.
- **Think of them as:** The **tools** you bring with you to the destination.
- **Example:**

```
dir /s
```

➤ *"Include subfolders when listing files."*

🔧 Switch = tools you bring with you.

✅ Put Together:

```
dir D:\Downloads /s
```

➤ *"List everything in Downloads, including all its subfolders."*

WHY USE BATCH FILES? 📁

- **Batch files** are still useful for automating repetitive tasks
- They save you from memorizing every command and switch
- Act as a **personal memory vault** for complex or frequently used command sequences
- Write commands once in a .bat file, then run it anytime via double-click or the command line
- Ideal for quick automation without writing full scripts or programs

📌 What Is a Batch File, technically?

- A **batch file** is just a plain text file with the extension **.bat** or **.cmd**.
- It contains a list of command line commands that execute **in order**.
- You can open and edit them in any text editor (e.g., Notepad, Notepad++).

📌 Real-World Example

Let's say you have to check the running processes on a remote computer every morning.

Typing this out every time:

```
tasklist /S 192.168.1.10 /U Admin /P *
```

But you can write a batch script:

```
tasklist1.bat X
tasklist1.bat
1 @echo off
2 echo Checking remote process...
3 tasklist /S 192.168.1.10 /U Admin /P *
4 pause
5
```

Now every time you run that file, it:

- **Runs the command.**
- **Prompts for the password.**
- **Pauses so you can read the output.**

Boom. 2 seconds of work.

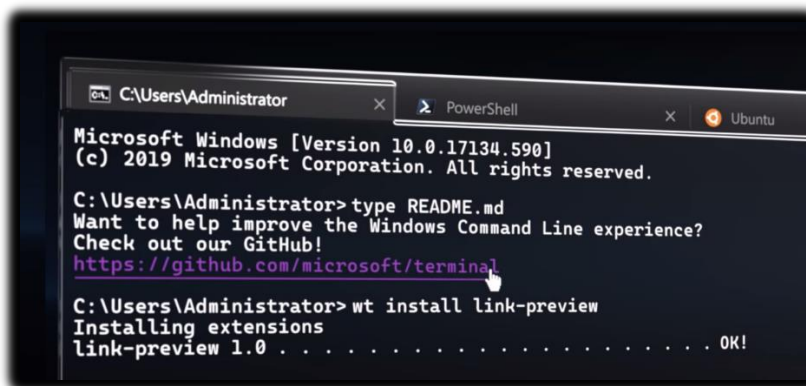
What Makes Batch Files Special?

- **No need for fancy compilers or runtimes.** They run directly through cmd.exe.
- **Flexible.** You can add logic like IF, GOTO, and FOR to make them dynamic.
- **Portable.** You can carry them on a flash drive and run them on any Windows machine.



⚠ Gotchas to Keep in Mind

- **Powerful but not secure:** anyone can open a batch file and see stored commands or passwords
- Use BitLocker or other encryption if you're storing sensitive info in batch files
- **Limited error handling:** failures mid-execution may not provide clear feedback
- **Not ideal for complex tasks:** for long-term scripting or interactive tasks, PowerShell or Python is better



✖ Coming Soon: Chapter 7 – Deep Dive into Batch Files

That chapter will show you:

- How to use variables in batch files.
- Conditional logic (yes, IF statements).
- Looping over files or input.
- Making them interactive (e.g., user input).

But we ain't there yet.

This section plants the seeds.

Start thinking about commands, repeat, repeat, then start writing batch files.

💻 Writing Scripts — What's the Difference?

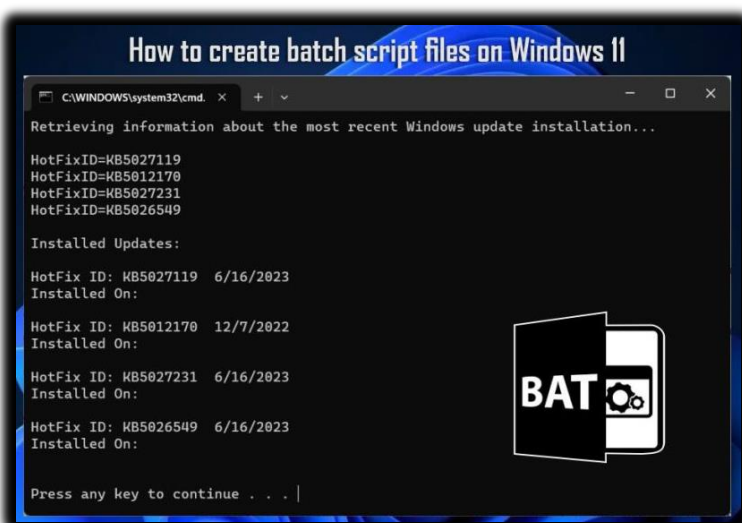
Scripts are like batch files' smarter older siblings.

They're still not real applications, but they can:

- Handle **more logic** (loops, conditionals, variables)
- Access **Windows APIs** or features directly (e.g., WMI, Active Directory)
- Work with objects, not just raw text or command output
- Be reused and maintained more easily

Batch files? Great for quick automation.

Scripts? Great when you want the OS to actually help you do smarter stuff.



```
How to create batch script files on Windows 11
C:\WINDOWS\system32\cmd. x + v
Retrieving information about the most recent Windows update installation...
HotFixID=KB5027119
HotFixID=KB5012170
HotFixID=KB5027231
HotFixID=KB5026549

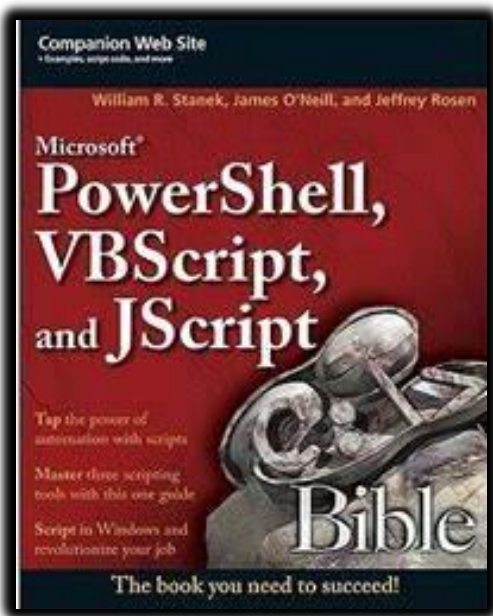
Installed Updates:
HotFix ID: KB5027119 6/16/2023
Installed On:
HotFix ID: KB5012170 12/7/2022
Installed On:
HotFix ID: KB5027231 6/16/2023
Installed On:
HotFix ID: KB5026549 6/16/2023
Installed On:

Press any key to continue . . . |
```

What's a Script? 💡

A **script** is a file written in a **scripting language** that gets run by a **script interpreter**. Windows ships with interpreters for:

- VBScript (Visual Basic Scripting Edition): **.vbs**
- JScript (Microsoft's version of JavaScript): **.js**
- PowerShell (The king now): **.ps1**



Each of these lets you do more than batch files ever could. Think:

- Reading from files.
- Creating GUIs.
- Interacting with Windows services.
- Querying network info, system health, etc.

What You Get from Scripting (vs Batch)

Feature	Batch File (.bat)	Script (e.g., PowerShell .ps1)
Variables	Yes (limited)	Yes (advanced types, arrays, objects)
Conditional Logic	Yes	Yes (complex logic, <code>switch</code> , etc.)
Loops	Basic	Full <code>for</code> , <code>foreach</code> , <code>while</code> loops
GUI Support	No	Yes (via Windows Forms/WPF in PowerShell)
Windows Feature Access	Minimal (via utilities)	Deep access (registry, services, AD)
Error Handling	Minimal	Try/catch error handling

Script Interpreter

Unlike batch files, scripts **need an interpreter**:

- **.vbs** → Run with `wscript.exe` or `cscript.exe`
- **.ps1** → Run with `powershell.exe` or `pwsh` (PowerShell Core)
- **.js** → Also `cscript.exe` (yep, Microsoft's JScript engine)

Example (PowerShell script):

```
PS C:\Users\Nick The Chief> Get-Process | Where-Object { $_.CPU -gt 100 }
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
627	33	138500	182208	121.84	4444	0	aswEngSrv
1087	30	72216	89564	177.44	1236	0	aswidsagent
4963	101	140832	129116	264.33	3588	0	AvastSvc
2406	54	39196	44792	192.25	9840	1	AvastUI
522	51	506572	580372	608.02	528	1	chrome
3557	82	144960	219192	380.67	7268	1	chrome
929	57	249732	191032	324.19	10600	1	chrome
534	46	215484	210868	318.44	12672	1	chrome
478	25	7712	29428	664.94	7148	1	Clover
1766	57	165628	94492	1,301.61	1528	1	dwm
5871	401	944208	310108	1,735.55	2436	1	explorer
649	39	131416	96328	329.94	4776	0	GWctlSrv
1003	69	357044	166756	678.98	796	1	msedge
460	37	28312	55252	145.17	5008	1	msedge
459	39	242052	185964	168.63	9352	1	msedge
625	29	199720	171228	202.94	9444	1	msedge
6970	214	167868	279496	461.41	10264	1	msedge
437	37	193196	203412	168.16	12748	1	msedge
1217	69	105976	135244	538.19	10344	1	ShareX
462	33	13780	21896	111.66	4044	0	svchost
9955	0	200	1528	547.47	4	0	System
865	38	51636	62028	120.30	9644	1	TextInputHost
1520	64	340192	239512	102.83	3024	1	WINWORD

Get a list of all the running processes, and then filter that list to show only the processes where the CPU usage is greater than 100.

- **Handles:** Number of references the process has to system resources (files, registry keys, etc.); high numbers may indicate heavy usage or a resource leak
- **NPM(K) – Non-Paged Memory (KB):** Memory allocated that **must stay in RAM** and cannot be paged to disk
- **PM(K) – Paged Memory (KB):** Memory allocated that **can be written to disk** if RAM is needed
- **WS(K) – Working Set (KB):** Physical RAM currently used by the process; subset of paged and non-paged memory actively in use
- **CPU(s):** Total CPU time (in seconds) the process has used since starting
- **Id (PID):** Unique Process Identifier assigned by the OS to manage the process
- **SI – Session ID:** Indicates which session the process is running in (e.g., session 0 = system services, others = user accounts)
- **ProcessName:** Name of the executable that started the process (e.g., chrome.exe, explorer.exe, AvastSvc.exe)

Why Scripts Matter

- Need to **query Active Directory** for disabled accounts? Script it.
- Want to **clean up old temp files** across network shares? Script it.
- Automate **software installs, audits, backups, or reports?** Script it.

You get the idea: when batch files just ain't cutting it, scripts are the next logical step.

Sample VBScript:

```
file1.vbs
1  ' VBScript Example: Displaying System Information and Creating a File
2
3  Option Explicit ' Forces explicit variable declaration
4
5  Dim objWMIService, colItems, objItem, objFSO, objFile
6
7  Set objWMIService = GetObject("winmgmts:\\.\root\cimv2")
8  Set colItems = objWMIService.ExecQuery("SELECT * FROM Win32_OperatingSystem")
9
10 For Each objItem In colItems
11     WScript.Echo "Operating System: " & objItem.Caption & " " & objItem.Version
12 Next
13
14 Set objFSO = CreateObject("Scripting.FileSystemObject")
15 Set objFile = objFSO.CreateTextFile("C:\temp\vbs_info.txt", True)
16 objFile.WriteLine("VBScript System Info")
17 objFile.Close
18
19 WScript.Echo "File created: C:\temp\vbs_info.txt"
```

Sample Jscript – similar syntax to Js:

```
JS file1.js > ...
1  // JScript Example: Reading Environment Variables and Displaying a Message
2
3  var shell = new ActiveXObject("WScript.Shell");
4  var env = shell.Environment("PROCESS");
5  var username = env("USERNAME");
6
7  WScript.Echo("Hello, " + username + "!");
8
9  var fso = new ActiveXObject("Scripting.FileSystemObject");
10 var file = fso.CreateTextFile("C:\\temp\\jscript_info.txt", true);
11 file.WriteLine("JScript Environment Info: User - " + username);
12 file.Close();
13
14 WScript.Echo("File created: C:\\temp\\jscript_info.txt");
15
16 try {
17     var version = shell.RegRead("HKLM\\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\ProductName");
18     WScript.Echo("Windows Version: " + version);
19 } catch (e) {
20     WScript.Echo("Error reading registry: " + e.message);
21 }
```

Sample powershell script:

```
> file1.ps1
1 # PowerShell Example: Getting System Info and Creating a File
2
3 $OS = Get-WmiObject Win32_OperatingSystem
4 Write-Host "Operating System: $($OS.Caption) $($OS.Version)"
5
6 $Env:USERNAME | Out-File -FilePath "C:\temp\powershell_info.txt"
7 "PowerShell System Info" | Add-Content -Path "C:\temp\powershell_info.txt"
8 Write-Host "File created: C:\temp\powershell_info.txt"
9
10 Get-Process | Where-Object {$_.CPU -gt 1} | Select-Object Name, CPU, Id | Format-Table -AutoSize
11 Get-Service | Where-Object {$_.Status -eq "Running"} | Select-Object Name, Status | Format-List
12
13 try {
14 | Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion" | Select-Object ProductName
15 } catch {
16 | Write-Error "Error reading registry: $($_.Exception.Message)"
17 }
```

Command Line

These images show us something. This means:

- The **term “command line”** is broader than just cmd.exe
- Includes **traditional batch files (.bat)**, **VBScript**, **JScript**, and **PowerShell**
- Encompasses any text-based interface where you type commands or scripts to perform tasks
- Covers automation, scripting, and direct system control beyond the GUI

It refers to **text-based interfaces** for interacting with the OS, as opposed to GUI tools

- **cmd.exe (Command Prompt):**

- ✚ Traditional Windows CLI
- ✚ Runs built-in commands and batch files (.bat, .cmd)

- **PowerShell:**

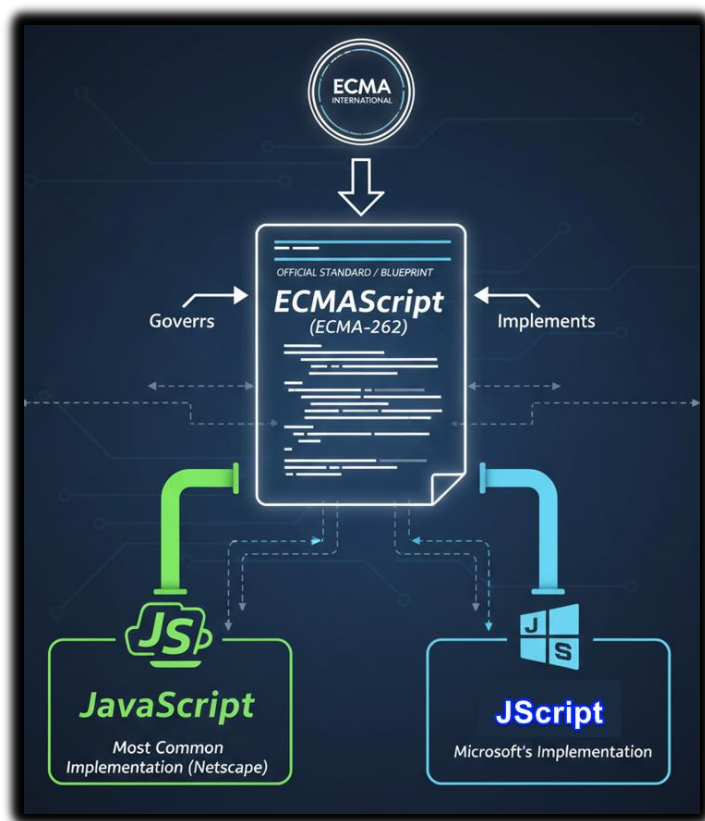
- ✚ Advanced shell and scripting language
- ✚ Works with **objects** instead of just text
- ✚ Uses **cmdlets** and supports sophisticated scripts

- **JScript & VBScript:**

- ✚ Microsoft scripting languages
- ✚ Run from CLI using jscript.exe, cscript.exe, or wscript.exe
- ✚ Part of Windows’ broader “command-line ecosystem”

JScript vs JavaScript

- **ECMAScript** → the official blueprint or standard for the scripting languages. This means If I want to create a language like JS or Jscript, I will have to use the official EcmaScript documentation called **ECMA-262**.
- **JavaScript** → the most common implementation of that standard (originally by Netscape)
- **JScript** → Microsoft's implementation of the same standard
- Differences may appear in:
 - ✚ Extra features Microsoft added to JScript
 - ✚ Slight compatibility variations
 - ✚ Browser usage: JavaScript dominates; JScript was mostly for IE and server-side ASP



Coming in Chapters 8 & 9

- **Chapter 8:** Intro to scripting (VBScript or PowerShell)
- **Chapter 9:** Active Directory scripting and automation in domain environments

COMMAND-LINE UTILITIES BY PURPOSE

1. Windows CLI: Data-Specific Commands

- Designed for **direct manipulation of data and the file system**
- Handle tasks such as:
 - ✚ File creation, deletion, and modification
 - ✚ Directory management
- Essential for **automation** of repetitive tasks
- Help maintain **consistent system configurations**



Examples:

- **dir**: Lists files and directories.
- **copy, xcopy**: Copies files and directories.
- **del, erase**: Deletes files.
- **mkdir, rmdir**: Creates and removes directories.
- **sort**: Sorts file contents.
- **type**: Displays file contents.
- **print**: Prints files.

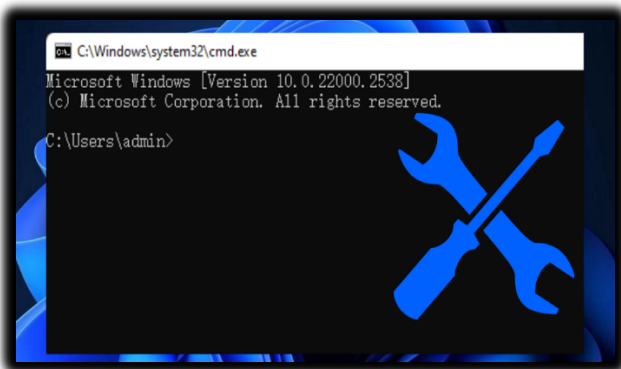
Use Cases:

- Automating user setup: Creating batch scripts to configure user environments quickly and consistently.
- Archiving data: Using batch files to automate file archiving processes.
- Bulk file operations: Performing mass file manipulations efficiently.
- Creating automated backups.

2. System Status Commands:

Purpose:

- These commands provide information about the system's hardware and software configuration.
- They are crucial for diagnosing problems, monitoring system performance, and managing hardware.
- They allow for remote system management, reducing the need for physical access.



Examples:

- **systeminfo**: Displays detailed system configuration.
- **tasklist**: Lists running processes.
- **driverquery**: Lists installed device drivers.
- **wmic**: Windows Management Instrumentation Command-line, for system management.
- **typeperf**: performance monitoring.
- **dxdiag**: DirectX diagnostics.

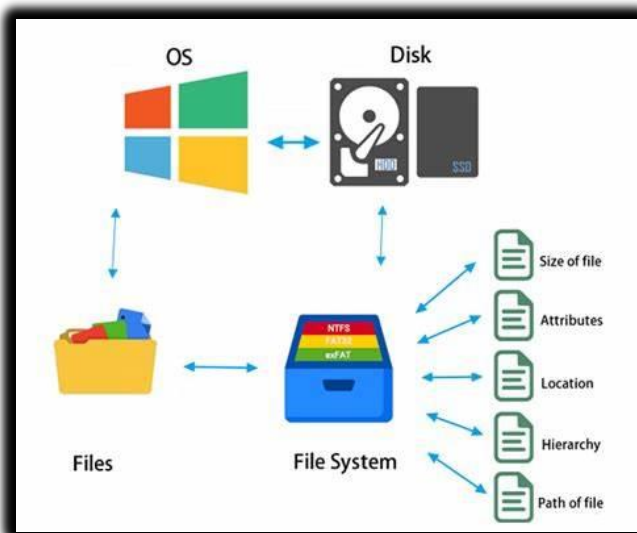
Use Cases:

- Troubleshooting hardware and software issues.
- Monitoring system performance and resource usage.
- Remote system administration and diagnostics.
- Creating automated performance logs and alerts.

3. File and Resource Management Commands:

Purpose:

- These commands manage files and system resources, including hardware and services.
- They provide tools for manipulating file content, managing power settings, and controlling services.
- They are essential for system optimization and resource allocation.



Examples:

- **findstr**: Searches for strings within files.
- **powercfg**: Manages power configuration settings.
- **net start, net stop**: Manages services.
- **netsh**: Network shell, used to configure network devices.

Use Cases:

- Searching for specific data within files.
- Optimizing power consumption.
- Managing network services and configurations.
- Automating service management.

4. Security and System Monitoring Commands:

Purpose:

- These commands focus on monitoring and improving system security.
- They help detect potential security threats and maintain system integrity.
- They are vital for proactive security management and intrusion prevention.



Examples:

- **ipconfig:** Displays network configuration.
- **netstat:** Displays network connections and statistics.
- **auditpol:** Manages security audit policies.
- **whoami:** Displays user information.
- **gpresult:** Displays group policy settings.

Use Cases:

- Monitoring network activity for suspicious connections.
- Auditing user activity and system events.
- Configuring security policies and access controls.
- Troubleshooting network connectivity issues.

5. Developer and Low-Level Task Commands:

Purpose:

- These commands provide low-level access to system resources and are often used by developers and advanced users.
- They can perform critical system operations, such as disk partitioning and DLL management.
- They require caution and expertise due to their potential impact on system stability.



Examples:

- **diskpart**: Manages disk partitions.
- **sfc**: System File Checker, repairs system files.
- **regsvr32**: Registers and unregisters DLLs.
- **shutdown**: Shuts down or restarts the system.

Use Cases:

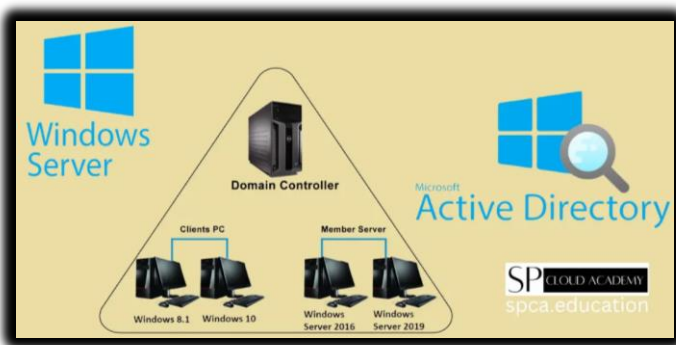
- Disk partitioning and management.
- System file repair and integrity checks.
- DLL registration and troubleshooting.
- Automated system shutdowns and restarts.

These are powerful ones for system administration and automation.

1. Active Directory (AD) Commands:

Purpose:

- These commands are specifically designed for managing Active Directory environments.
- They automate tasks related to user accounts, groups, organizational units (OUs), and other AD objects.
- They are essential for enterprise-level system administration, ensuring consistency and efficiency.



Active Directory Components - Active Directory enables consistent, centralized **network administration** and **resource management**

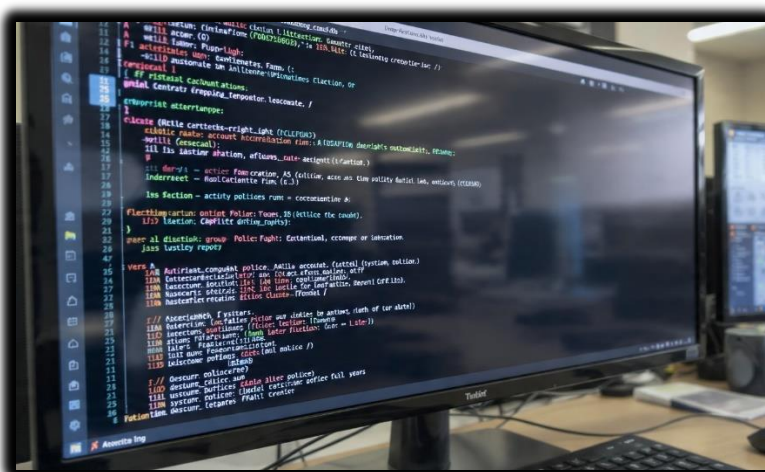
- **Domain Controller (DC):**
 - ✚ Central component of Active Directory
 - ✚ Stores the directory database
 - ✚ Provides authentication and authorization services within the domain
- **Windows Server:**
 - ✚ Operating system hosting Active Directory
 - ✚ Examples: Windows Server 2016, 2019, 2022 (latest supported versions)
- **Clients and Member Servers:**
 - ✚ Client PCs (e.g., Windows 10, Windows 11) and member servers connect to the DC
 - ✚ AD manages resources, users, and permissions across the network domain
 - ✚ Illustrates core AD functions: centralized management and access control

Key Concepts:

- Automation: AD commands are often used within scripts to streamline repetitive tasks.
- Consistency: Scripts ensure uniform configuration across the AD environment.
- Reliability: Automated tasks reduce human error and improve reliability.

Use Cases:

- User account creation and management.
- Group policy administration.
- OU management.
- Automated reporting and auditing.
- Automating user onboarding, offboarding, and group management.



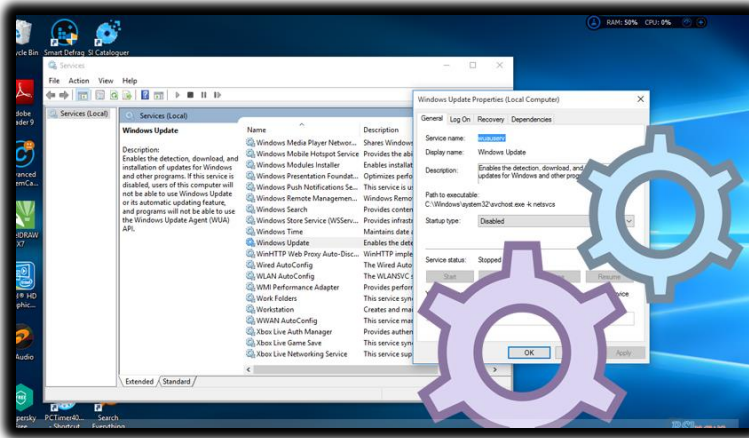
Example Scenarios:

- Creating a script to automatically provision new user accounts with specific attributes and group memberships.
- Using commands to generate reports on user account activity or group memberships.
- Modifying group policies across many computers.

2. Services Commands:

Purpose:

- Services are background applications that run without direct user interaction.
- Service commands allow you to manage these services, including starting, stopping, and configuring them.
- Effective service management is crucial for system performance and security.



Key Concepts:

- Resource Management: Unnecessary services can consume system resources.
- Security: Some services can create security vulnerabilities.
- Automation: Batch scripts can automate service management tasks.

Use Cases:

- Starting and stopping services as needed.
- Disabling unnecessary services to improve performance.
- Automating service configuration during system startup or shutdown.
- Automating services for development environments.

Example Scenarios:

- Creating a batch script to start development-related services when needed.
- Disabling potentially vulnerable services when not in use.
- Automating service restarts after system updates.

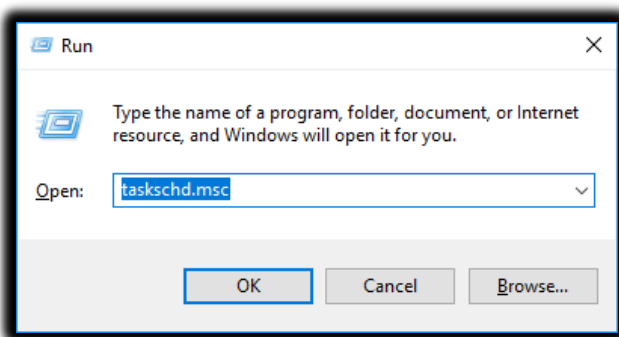
Important Considerations:

- Service names can vary between Windows versions.
- Incorrect service management can lead to system instability.

3. Task Scheduling Commands:

Purpose:

- Task scheduling allows you to automate the execution of commands and scripts at specific times or events.
- It's essential for automating routine maintenance tasks and ensuring timely execution of critical processes.
- It allows for background task automation.



Key Concepts:

- **Automation:** Scheduling tasks eliminates the need for manual intervention.
- **Efficiency:** Tasks can be run during off-peak hours to minimize impact on system performance.
- **Reliability:** Scheduled tasks ensure that critical processes are executed consistently.

Use Cases:

- Automating backups and system maintenance.
- Running scripts at scheduled intervals.
- Generating reports and logs automatically.
- Automating after hours scripting.

Example Scenarios:

- Scheduling a daily backup script to run at midnight.
- Automatically generating system performance reports on a weekly basis.
- Automating log file rotation.
- Automatically running virus scans.
- **schtasks**: The primary command-line tool for managing scheduled tasks.

Summary:

- Low-level commands require careful handling due to their potential impact.
- Remote management is made much easier with the command line.
- Active Directory commands are crucial for enterprise system administration.
- Service management is essential for system performance and security.
- Task scheduling automates routine processes, improving efficiency and reliability.

MICROSOFT UTILITIES UPDATES

Updating Your Utilities – Microsoft Download Center

- Command line tools can get outdated, but Microsoft provides updated versions on the **Download Center**
- The Download Center is separate from **Windows Update** and hosts tools Microsoft doesn't push automatically
- **Microsoft Download Center**: hidden catalog of updated utilities and tools

Navigating the Site

- **Favorites at the Top**: homepage highlights popular tools and current patches
- **Search by Technology**: search by category like “Windows Server tools,” “PowerToys,” “Sysinternals” for better results
- **Search by Keyword**: use the keyword search form for specific tools

Pro Tip Hack: Google Advanced Search

- Use Google if Microsoft search is unreliable
- “All these words” → type tool/utility name (e.g., Process Explorer)
- “Site or domain” → www.microsoft.com

Explore by Category

- Useful if you don’t know the exact name or are just browsing
- Categories like “Developer Tools,” “System Utilities,” or “Command Line Tools” help discover:
 -  Windows Terminal
 -  PowerToys

Third-Party Utilities = Power Tools

Supercharging the Command Line

- Default **Windows CMD** = basic, like a plastic butter knife
- **Advanced tools (“chainsaws”):**
 -  **Sysinternals Suite:** Process Explorer, Procmon, PsExec, Autoruns — essential for reverse engineering and malware analysis
 -  **PowerToys:** Modern tweaks — window snapping, keyboard shortcuts, bulk renaming
 -  **7-Zip Command Line:** Fast, scriptable archiving and extraction
 -  **Windows Terminal:** Tabs, themes, UTF-8, WSL integration
 -  **Chocolatey / Scoop:** Command-line package managers for fast tool installation

Reverse Engineering Tie-In

- Using Sysinternals elevates you to advanced CMD user level
- Procmon and similar tools are mandatory for observing malware behavior
- Built-in commands are fine, but **real power comes from these extra utilities**
- Treat CLI like a development environment — comfortable, efficient, responsive
- Pro tip: block cursor + full-screen = distraction-free sessions

• Command History

Buffer Size

Set high to allow scrolling through long command sessions without losing history.

999 – 4096

Number of Buffers

Determines how many separate process histories are maintained simultaneously.

4 Buffers



QuickEdit Mode

Drag-to-select

Highlight text directly. Combined with **Right-Click** to paste, it's the fastest way to move data.

When to Disable?

Only for legacy full-screen apps (like old terminal games) that need to capture mouse clicks.



Insert Mode

Non-destructive Pasting

Enables pasting at the cursor without eating the characters already on the line.

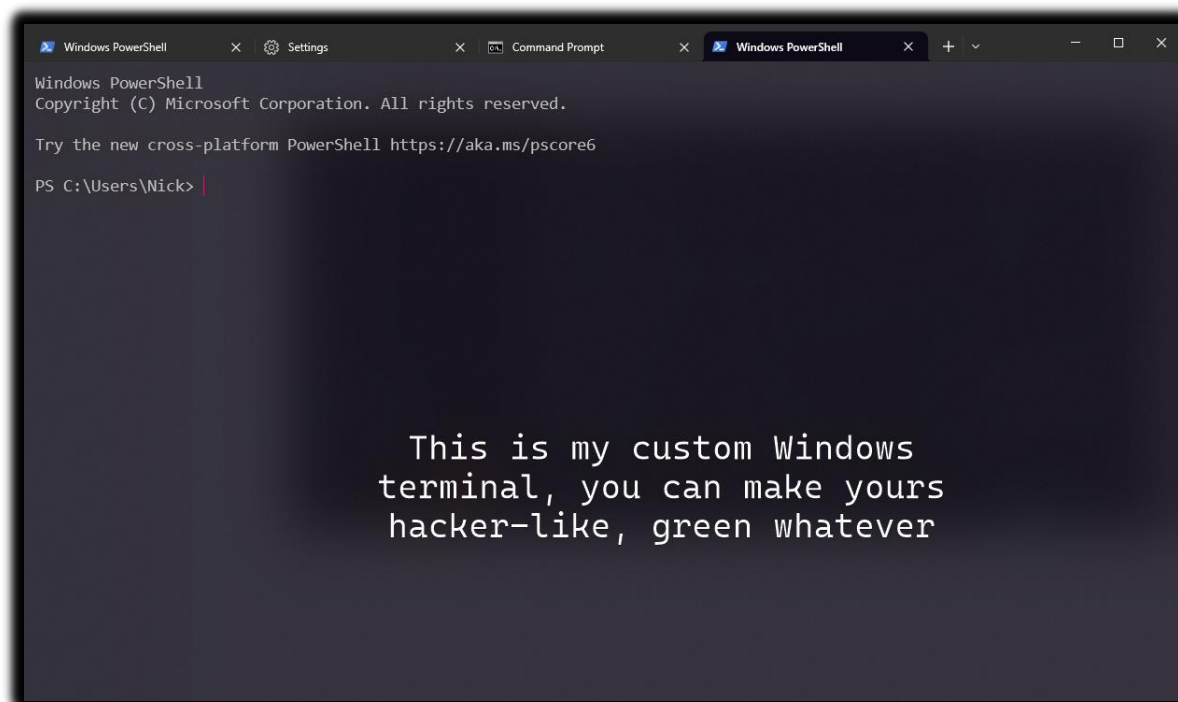
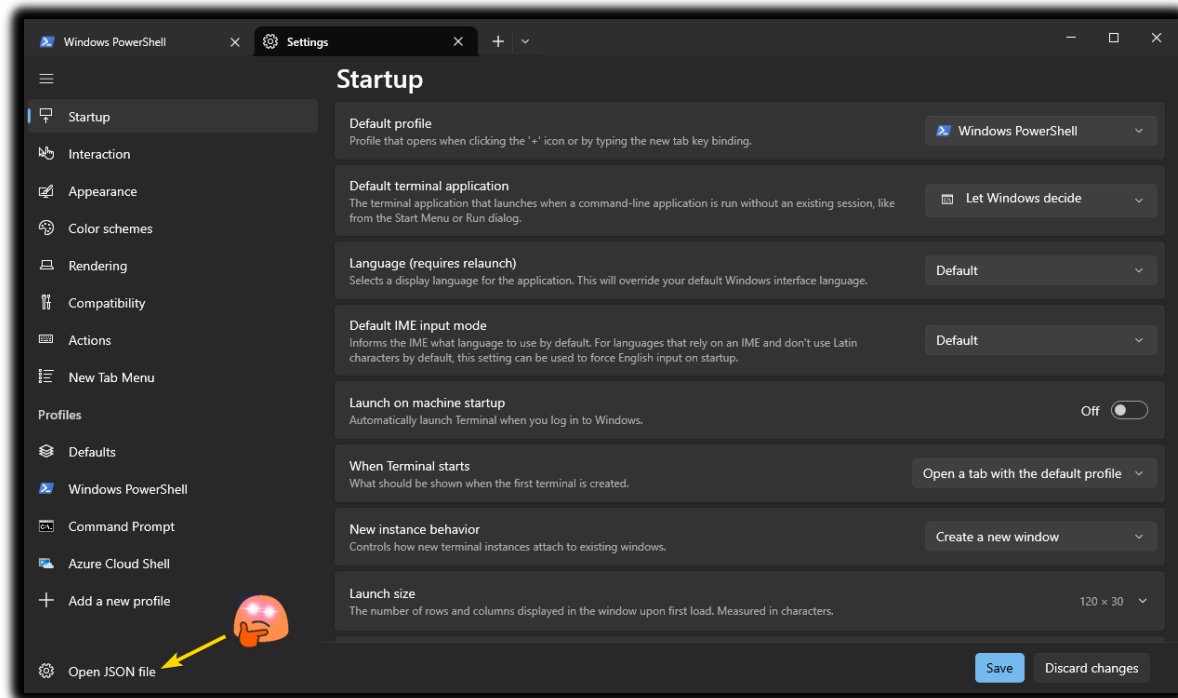
Best Use Case

Injecting **long/file/paths** or complex commands into the middle of an existing string.

💡 *Tip: Keeps your input clean and safe.*

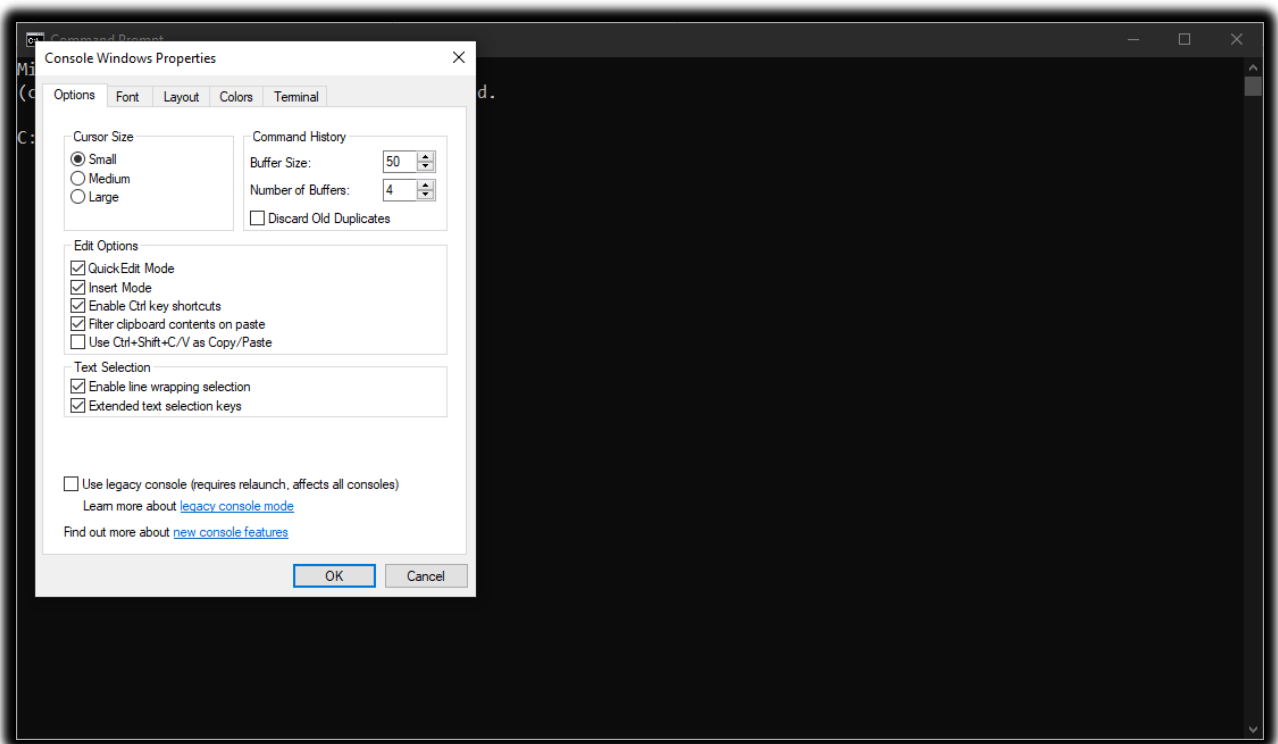
Modern Bonus – Windows Terminal Profiles

You can go next level by setting different profiles and changing your windows terminal to look any way you like in the **settings json file**:



QUICK EDIT MODE

- Enables **mouse-driven copy and paste** in CMD
- Makes the CMD window more usable with **drag-to-select** and **right-click paste**
- Eliminates the need for old-school Ctrl+C/Ctrl+V hacks
- Right-click the CMD title bar → Properties → Options tab → check QuickEdit Mode
- Disable only if using **legacy full-screen apps** that require direct keyboard input



⚡ What QuickEdit Mode Does:

1. **Select text with your mouse** just by dragging (like you would in any normal text editor).
2. **Right-click to copy** the selected text.
3. **Right-click again to paste** it at the command prompt.

Understanding Internal Commands (CMD vs Utilities)

I. CMD.EXE = the Command Line Host

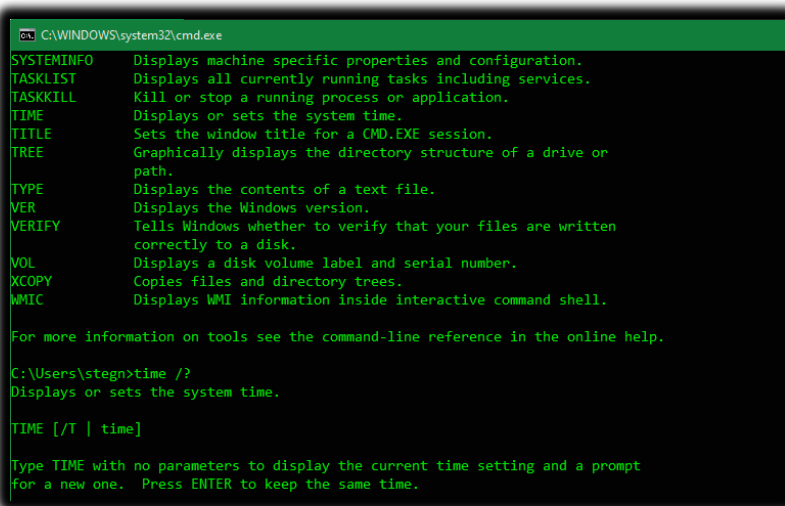
- When you run cmd, you're launching CMD.EXE, the shell that processes your commands.
- It stays open, reading your input, and running instructions directly.

II. Internal Commands (Built-In to CMD.EXE)

- These are baked into CMD.EXE itself.
- They don't exist as standalone .exe files.
- You can't find them on disk by running dir — they live inside the CMD process.

Examples:

- dir
- copy
- del
- cd
- echo
- set
- if, for, goto (used in batch scripting logic)



```
C:\WINDOWS\system32\cmd.exe
SYSTEMINFO    Displays machine specific properties and configuration.
TASKLIST      Displays all currently running tasks including services.
TASKKILL      Kill or stop a running process or application.
TIME          Displays or sets the system time.
TITLE         Sets the window title for a CMD.EXE session.
TREE          Graphically displays the directory structure of a drive or
              path.
TYPE          Displays the contents of a text file.
VER           Displays the Windows version.
VERIFY        Tells Windows whether to verify that your files are written
              correctly to a disk.
VOL           Displays a disk volume label and serial number.
XCOPY         Copies files and directory trees.
WMIC          Displays WMI information inside interactive command shell.

For more information on tools see the command-line reference in the online help.

C:\Users\stegn>time /?
Displays or sets the system time.

TIME [/T | time]

Type TIME with no parameters to display the current time setting and a prompt
for a new one. Press ENTER to keep the same time.
```

III. Utilities (External Commands)

- These DO exist as .exe files on your system.
- You can find them using dir, run them directly, or even from PowerShell or a script.

Examples:

- tasklist.exe
- ipconfig.exe
- ping.exe
- telnet.exe
- robocopy.exe

Located in: **C:\Windows\System32** or similar system paths.

These are referred to as **utilities** in the book.

Some utilities (like **Telnet**) create their own mini environments inside CMD.

Commands you type *inside* Telnet don't work outside of it — they're not in CMD, they're specific to that tool.

🔍 *Just like dir is internal to CMD, Telnet has its own internal commands once you're inside it.*

When you type something in CMD:

CMD.EXE checks: is this an internal command?

- **If yes** → run it directly
- **If no** → look for a matching .exe file (utility)

USER ACCOUNT CONTROL, RUNAS

UAC in Windows 10/11: Offers more granular control than Vista's all-or-nothing approach

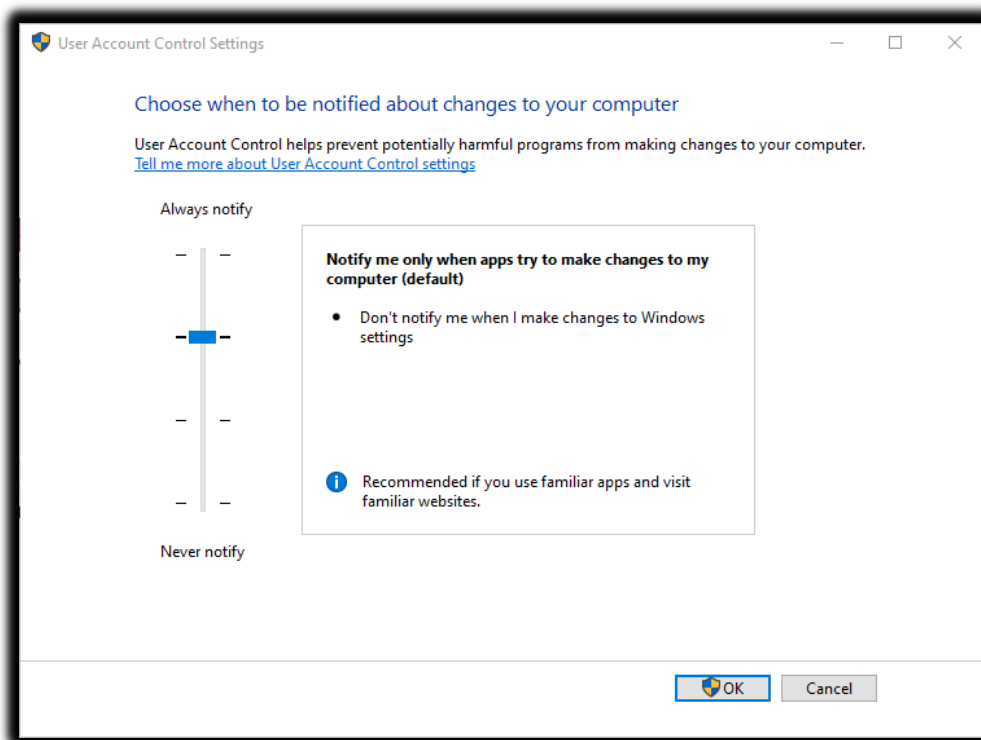
Users can choose notification levels: always notify → notify only when apps try changes (no desktop dim)

Prompts are **less frequent and less disruptive** for common admin tasks

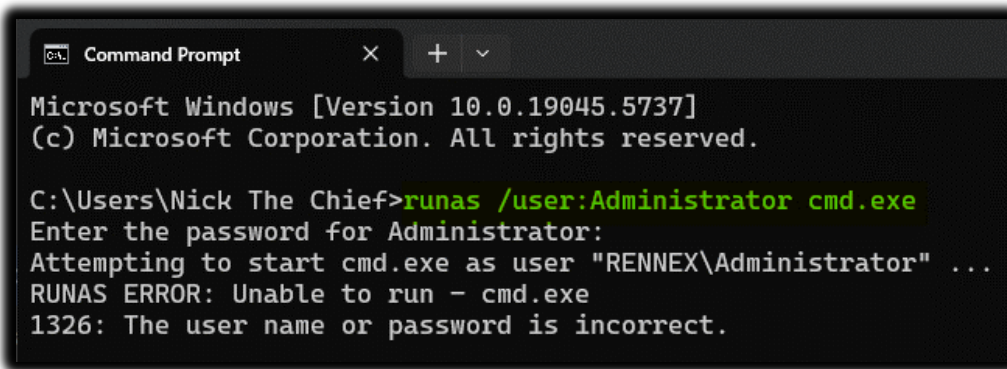
Purpose: prevent apps, scripts, or malware from performing admin-level tasks silently

Works with other security features: **Windows Defender Antivirus, Exploit Protection, robust permission models**

- Your **regular Windows login password** lets you sign in and run everyday apps
- The **Administrator account** has higher privileges than a normal user account
- Admin actions (installing software, changing system-wide settings) require **explicit approval**
- Even if you're logged in, Windows may ask for **admin credentials or confirmation**
- This separation prevents programs or malware from making system-level changes without permission



Runas Command for Elevation



```
Command Prompt
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Nick The Chief>runas /user:Administrator cmd.exe
Enter the password for Administrator:
Attempting to start cmd.exe as user "RENNEX\Administrator" ...
RUNAS ERROR: Unable to run - cmd.exe
1326: The user name or password is incorrect.
```

- runas lets you **run a command or program as another user** (usually an admin)
- It **will prompt for the admin password**
- runas **does NOT bypass UAC**
- It's used to **switch user context**, not disable security
- Useful for launching tools that need admin rights without logging out

Common Use Cases

- Run a tool as an admin from a standard user session
- Test how something behaves under a different account
- Launch an elevated CMD or PowerShell window

FEATURE	WINDOWS VISTA	WINDOWS 10 / 11
UAC Prompt	Frequent, aggressive; blocks even simple tasks.	Smarter prompts; usually triggers only when strictly needed.
File System Access	Locked-down System32 & Windows folder.	Still protected, but easier to access with proper elevation.
Network Drive Issues	UAC token separation often broke mappings.	Fixed via registry tweaks or Group Policy objects.
Local Security Policy	Required for making many basic changes.	Still valid, but less often needed due to better GUI options.
Default Admin Token	Severely limited, even for administrators.	Still limited, but Windows Terminal + Elevated CMD handles it smoothly.

Where UAC Fits In

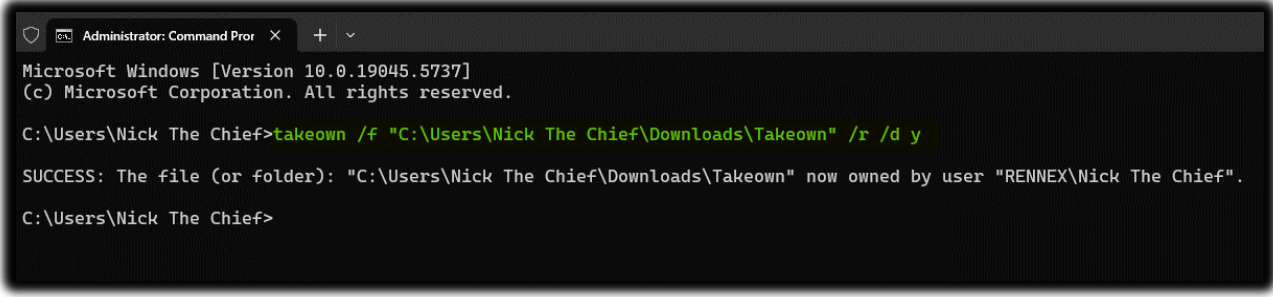
- UAC still applies after runas
- Admin approval is required for system-level changes
- You don't need to disable UAC to use runas
- That's it. Everything else is implementation detail and policy tuning—not required to *use* runas.

Modern Way to Give Yourself Permissions 🔥

If you need to access or modify protected folders (like **C:** or **C:\Windows**, etc.):

1. Take Ownership of the folder

Run an elevated CMD or PowerShell, then:



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

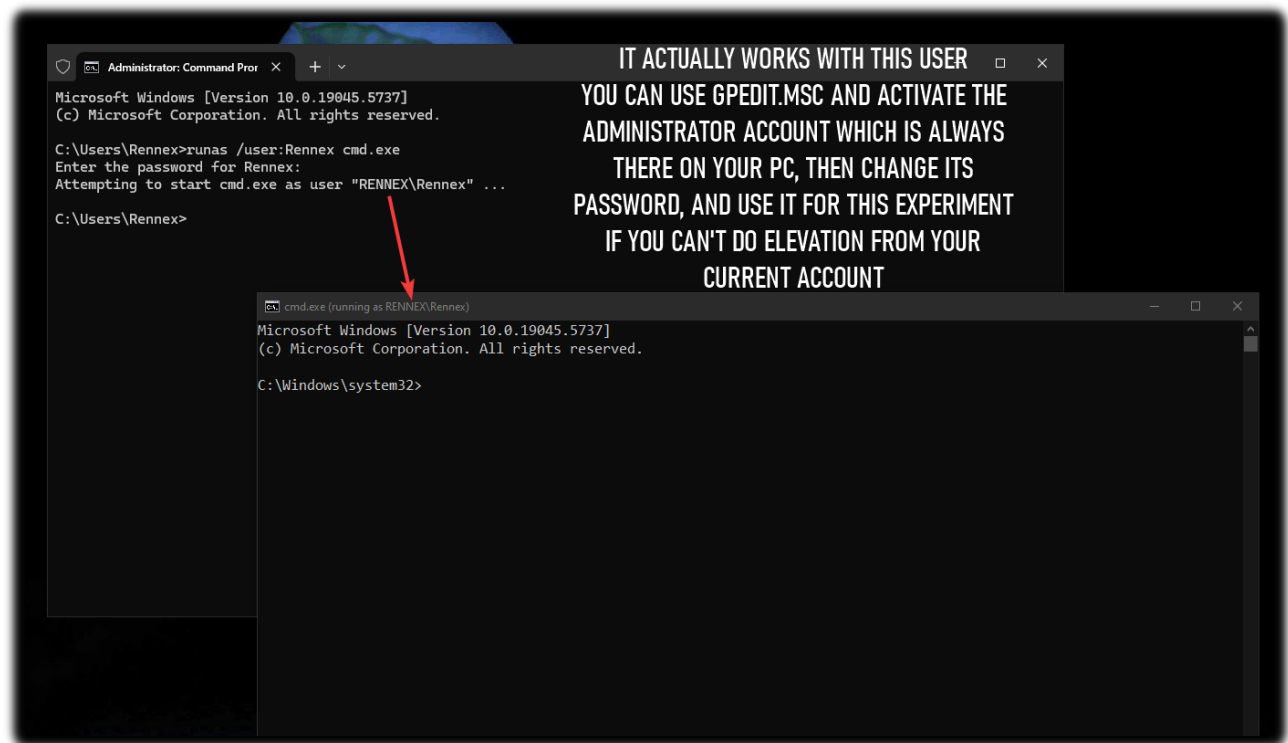
C:\Users\Nick The Chief>takeown /f "C:\Users\Nick The Chief\Downloads\Takeown" /r /d y

SUCCESS: The file (or folder): "C:\Users\Nick The Chief\Downloads\Takeown" now owned by user "RENNEX\Nick The Chief".

C:\Users\Nick The Chief>
```

✓ 2. Grant yourself full control

Still in the elevated terminal:



runas doesn't bypass UAC, but it's a good way to manually switch context if needed.

UAC Settings in Windows 10/11

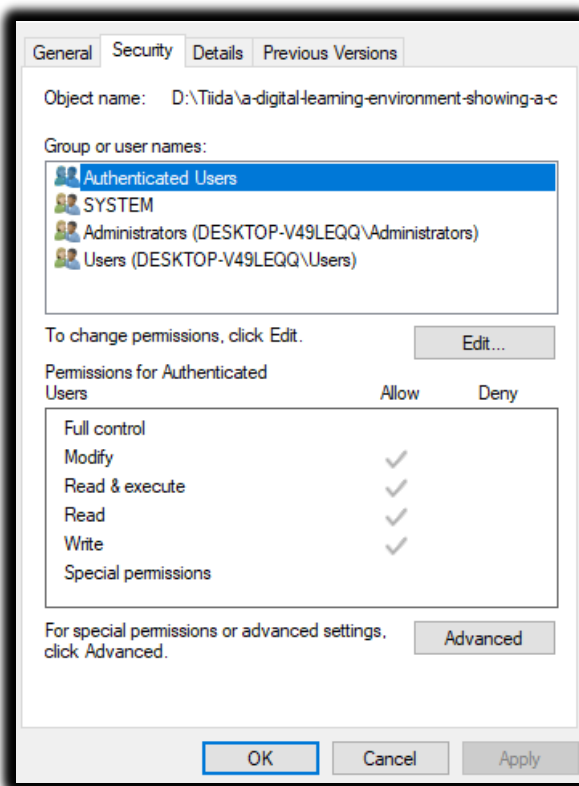
- Access via **Control Panel → System & Security → Security and Maintenance → Change User Account Control settings**; slider has 4 levels (default = level 2)
- **Level 1**: fewer prompts for dev/testing; **Level 0**: fully disabled (only in VM/lab)
- For finer control, use **Local Security Policy (secpol.msc)** to adjust:
 - Elevation prompt behavior
 - Admin Approval Mode for built-in Administrator
- **System folders (C:\Windows, System32)** remain protected; elevated shells or takeown/icacs needed for changes
- UAC is protective, not restrictive; run CMD/PowerShell as admin when needed.

🔑 **icacs** — Your God-Key to NTFS Permissions

icacs (introduced in XP SP2 / fully baked by Vista+) is the successor to **cacls** and **xcacs**, and it lets you:

- View and modify file/folder permissions
- Grant/revoke rights to users or groups
- Backup/restore access control lists (ACLs)
- Set inheritance rules
- Take ownership

Basically, it's your CLI way to do what the GUI "Security" tab does, but way more powerful when scripting or working fast.



View Permissions:

```
icac ls C:\MyFolder
```

```
C:\Users\Rennex\Downloads>icac ls C:\Users\Rennex\Downloads\Commandline\  
C:\Users\Rennex\Downloads\Commandline\ NT AUTHORITY\SYSTEM:(OI)(CI)(F)  
                                          BUILTIN\Administrators:(OI)(CI)(F)  
                                          RENNEX\Rennex:(OI)(CI)(F)
```

```
Successfully processed 1 files; Failed processing 0 files
```

```
C:\Users\Rennex\Downloads>
```

Grant Full Control to a User:

```
icac ls C:\MyFolder /grant Rennex:F
```

F = Full Control, you can also use:

- **M** = Modify
- **RX** = Read & execute
- **R** = Read
- **W** = Write

Remove a User's Permissions

```
icac ls C:\MyFolder /remove Rennex
```

Reset Permissions (very useful for fixing locked folders)

```
icacls C:\MyFolder /reset
```

Take Ownership

```
takeown /f C:\MyFolder /r /d Y  
icacls C:\MyFolder /setowner "YourUsername" /T
```

- Powerful stuff.
- Use with caution.
- One wrong move, and you'll nuked access for everyone.

Giving Yourself Permission (2026 Version)

Windows 10/11 tightened security **but gave you better tools to control it.**

Here's the real process, skipping Vista-era procedures:

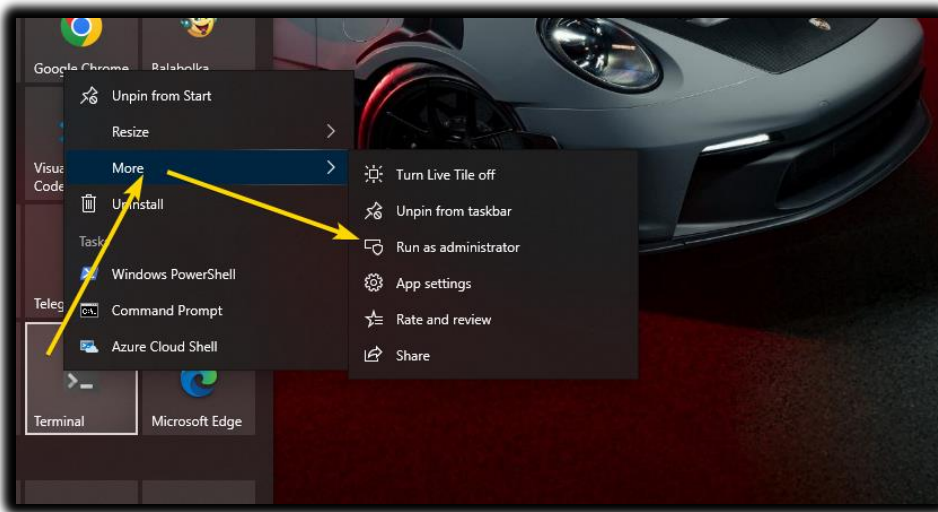
Step 1: Run Everything as Admin

Whether you're using:

- File Explorer
- CMD / PowerShell / Windows Terminal

 When editing protected system folders like **C:** or **C:\Program Files** etc.

Right-click → “Run as administrator.” (You can set it permanently via shortcut properties.)



Step 2: Take Ownership & Grant Yourself Access

When a folder like System32 or C:\Program Files blocks you, here's the proper flow.

Take Ownership:

```
takeown /f "C:\System32" /r /d Y
```

- `/f "C:\System32"` - Specifies the file or directory (in this case, the System32 folder)
- `/r` - Recursive, applies the operation to all files and subdirectories
- `/d` is a parameter that specifies the **default answer** to be used when a confirmation prompt would otherwise be displayed.
- **Y** means "Yes", so `/d Y` automatically answers "Yes" to any confirmation prompts.

This command is **taking ownership of the entire C:\System32 directory** and all its contents, automatically confirming any prompts that might appear during the process.

Grant Permissions:

```
icacls "C:\System32" /grant %USERNAME%:F /T
```

- **%USERNAME%** automatically inserts your current username
- **/T** = recursive through all subfolders and files
- **:F** = full control

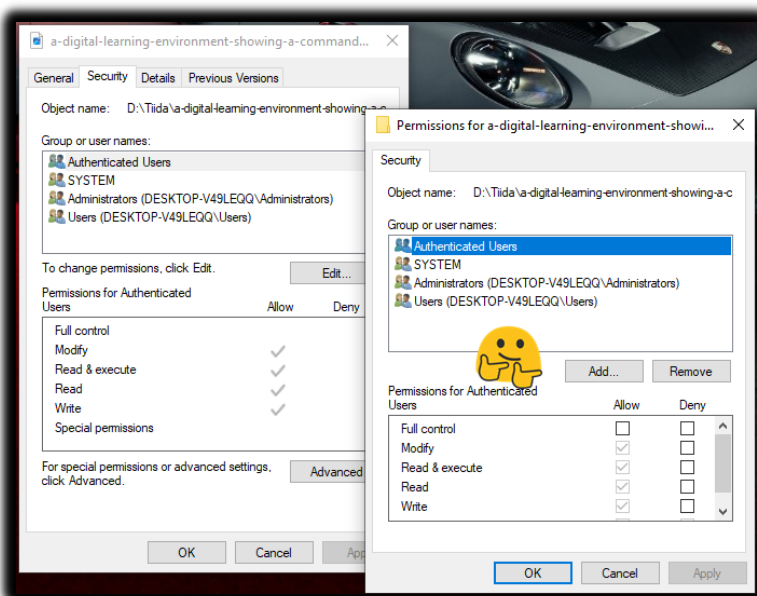
⚠ Be very careful doing this on **System folders** — one wrong move, and Windows might have a bad time. Use this on test VMs first.

Step 3: Re-enable Security (If You Temporarily Lowered UAC)

Pro Tip: Add Yourself to Folder Security GUI (Advanced Way)

1. Right-click folder → **Properties**
2. Go to **Security** tab → Click **Advanced**
3. Change Owner → Set to your user account
4. Then add your user to the list with full control.

⚠ Don't rely on "Administrators" group alone. **You must add your actual user account** for changes to stick and apply recursively.



💡 Bonus: Permissions No Longer “Flow Down” Like They Used To

Modern NTFS locks **don’t inherit the same way anymore** — so just giving yourself rights on C:\ doesn't unlock C:\Windows or C:\System32.

Each folder may have its own ACLs that must be **overridden individually**.

That’s why:

```
icacls C:\Windows /grant Rennex:F /T
```

...is separate from:

```
icacls C:\ /grant Rennex:F /T
```

Small confusion on my end. So icalcls is access control list? And it uses /T? Why not /r for recursive like takeown uses?

icacls = Integrity Control Access Control Lists

The name itself is lowkey geeky as hell:

- **i** = **Integrity** (for Mandatory Integrity Control stuff)
- **c** = **Control**
- **acls** = **Access Control Lists**

It’s basically **Microsoft showing off NTFS permissions**

Handles the key things you need to manage access:

- **ACLs** – who can do what
- **SIDs** – unique IDs for users and groups
- **Inheritance** – permissions passed down to files/folders
- **Ownership** – who controls a file or folder
- **Restrictions** – blocks actions you’re not allowed to do

Recursive flags in Windows are not all the same.

Using **takeown**, the **/r** flag makes the command recursive, meaning it will apply to the folder and all its subfolders and files.

This flag is specifically used to make yourself the owner of a file or folder and everything inside it.

```
takeown /f "C:\SecretFolder" /r /d Y
```

When using **icacls**, the **/T** flag makes the command recursive, applying permission changes to a folder and all its subfolders and files.

This is used for modifying permissions throughout an entire folder tree.

```
icacls "C:\SecretFolder" /grant %USERNAME%:F /T
```

Real-World Tip

If you're scripting or fixing broken access:

1. Take ownership first

```
takeown /f "C:\LockedStuff" /r /d Y
```

2. Then give yourself full permissions

```
icacls "C:\LockedStuff" /grant %USERNAME%:F /T
```

If you skip ownership, icacls might just say:

 *Access is denied.* 

What's the Deal with Zones on Network Drives? 🚨

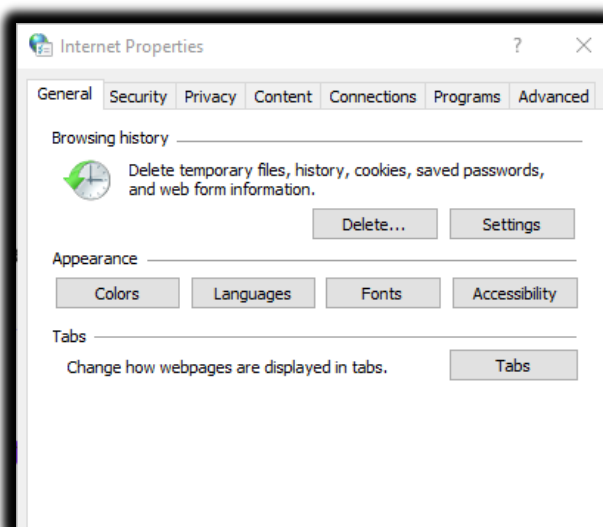
I. Network Drive Security Zones (Modern Windows)

- Windows assigns **security zones** to network drives, similar to Internet zones, to protect against malware
- Mapped drives (e.g., \\Server\SharedFolder) may default to the “**Internet**” zone, causing restrictions:
 - 🚧 Scripts are blocked
 - 🚧 Files may refuse to run
 - 🚧 Permissions can behave unexpectedly
 - 🚧 Drag-and-drop may be limited
- This can happen even on your **own machine acting as a server**
- **Fix / Workaround:** Add the network location to **Trusted Sites** in Internet Options (Zone Upgrade)
- Modern Windows handles this more gracefully than Vista, but some restrictions still apply for security reasons

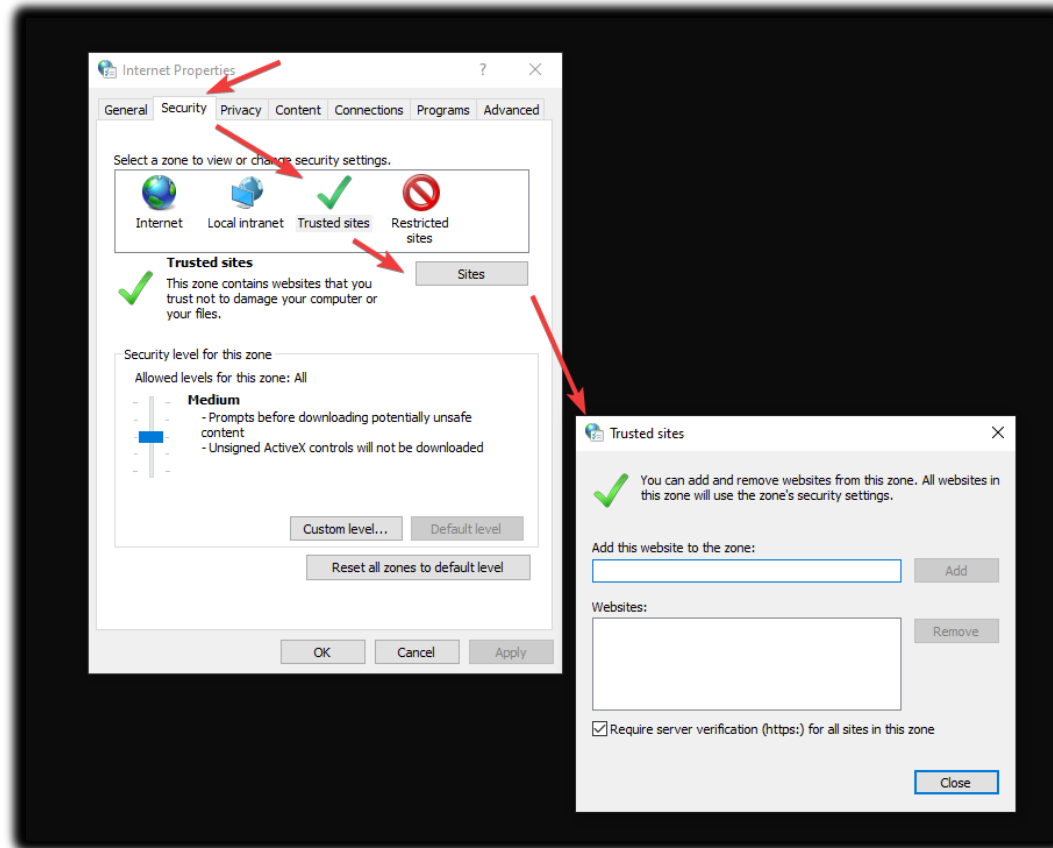
Open Internet Options - Start > Internet Options or run:

```
control inetcpl.cpl
```

The following appears:



Go to: Security tab → Click on Trusted Sites → Click Sites.



In the box, add your network drive's **UNC** (Universal Naming Convention) path:

`file://ServerName`

Or for IP-based access:

`file://192.168.1.100`

Don't put mapped drives like **Z:**.

It has to be the **actual network path** like **\\192.168.1.100\SharedFolder**

Uncheck the box: *"Require server verification (https:) for all sites in this zone"*

Otherwise, it might block your entry. Hit Add, Close, then OK.

Why This Works

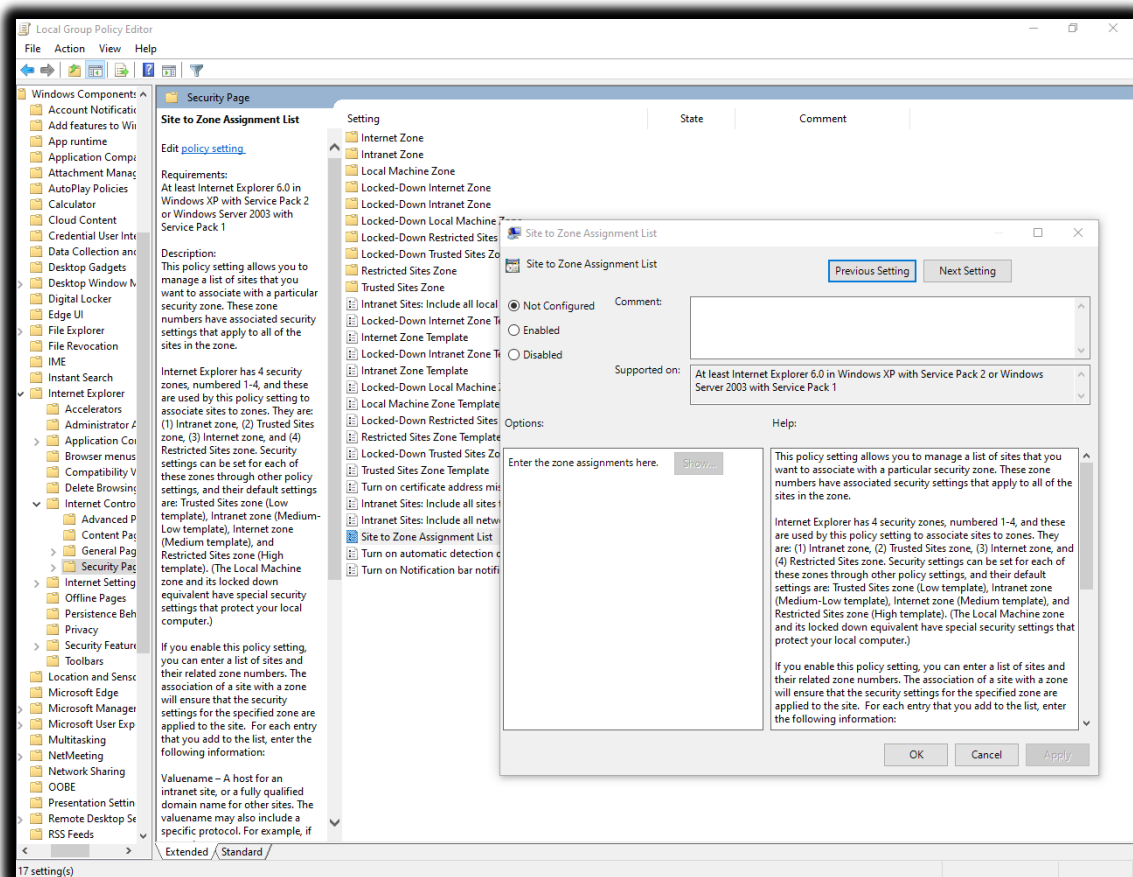
When you open files from the network, **they're subjected to Internet Explorer's security model** — yes, even in 2026.

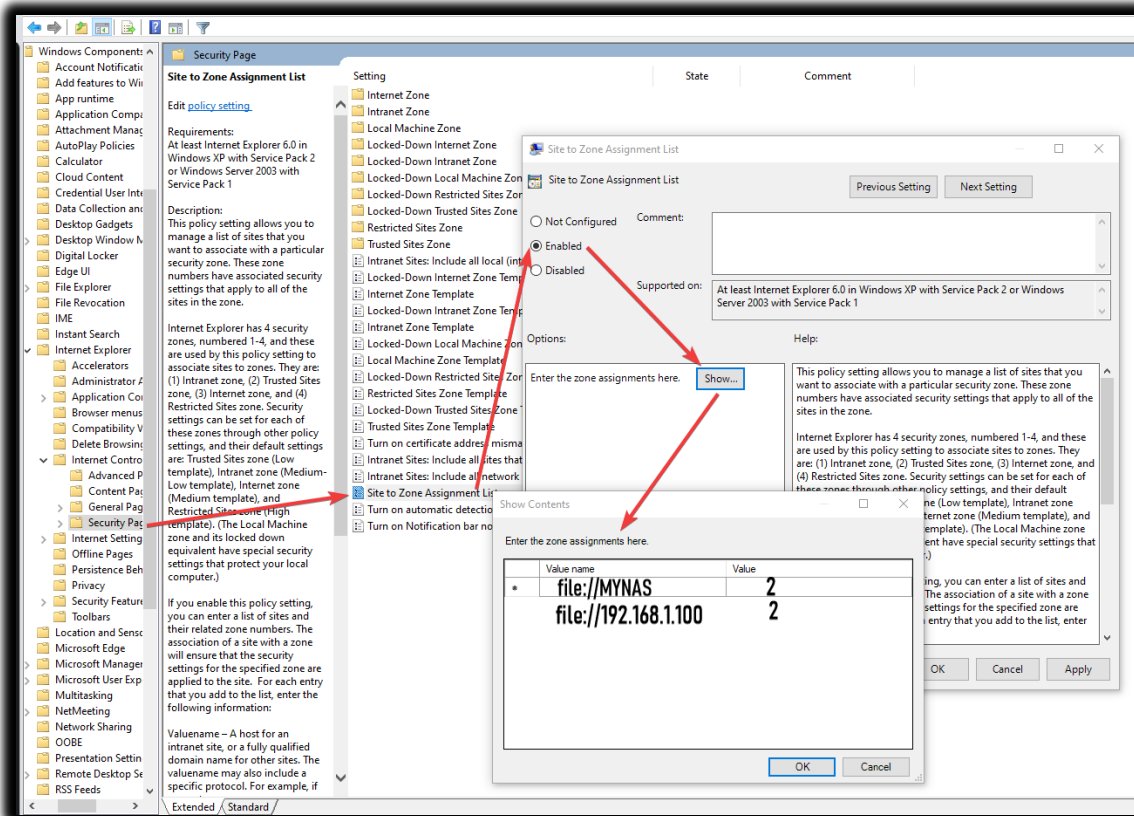
This Trusted Sites move **reclassifies** that location as **"safe"** so you:

- Avoid UAC nags
- Can run .bat, .exe, .ps1, .vbs off that network
- Avoid the *"This file came from another computer..."* warning
- And your scripts/policies won't fail silently

Pro-Tip with GPO (Group Policy)

If you're in a domain or want to **automate this on many machines**, you can do it via Group Policy: [User Configuration > Administrative Templates > Windows Components > Internet Explorer > Internet Control Panel > Security Page > Site to Zone Assignment List](#)





Result: Sites are treated as Trusted Sites automatically.

Vista Command Line Legacy (Modern Take)

- Most Vista-era command removals are **irrelevant today**
- Modern Windows uses updated tools (choice, bcdedit)
- Old DOS/16-bit commands are gone — ignore them

Batch Files Failing?

- Most failures are **permissions/UAC**, not bad logic
- Always test batch files **Run as Administrator** if they touch:

- Registry
- Program Files, Windows, System32
- Services or scheduled tasks

Key Takeaways for Modern Windows

- CMD is still powerful and relevant
- Learn **both CMD and PowerShell** — they complement each other
- UAC still blocks scripts silently without elevation
- Deprecated commands are gone for good — don't rely on them
- Windows doesn't push CLI usage — that's an advantage, not a flaw

Next Chapter Focus

- Core file commands: dir, xcopy, robocopy, del, attrib
- Command-line editing
- Registry interaction
- Backup and automation basics

🌟 Let's get it moving superstar!! 🌟 🙄 🙄